

涉密论文 ☐ 公开论文 ☒

浙 江 大 学

本科生毕业设计



题目	面向科学场景的智能体 工具调用优化研究
学生姓名	彭超琦
学生学号	3220105744
指导教师	朱霖潮
年级专业	2022级计算机科学与技术
所在学院	计算机科学与技术学院
递交日期	2026 年 5 月 18 日

毕业论文（设计）原创性声明

本人郑重声明：所呈交的毕业论文（设计）是本人在导师的指导下，严格按照学校和学院有关规定完成的，不存在学术不端行为。除文中特别加以标注和致谢的地方外，本论文（设计）不包含其他个人或集体已经发表或撰写过的研究成果，也不包含为获得浙江大学或其他教育机构的学位或证书而使用过的材料。对本研究做出贡献的个人和集体，均已在文中明确说明并表示谢意。

毕业论文（设计）作者签名：

签字日期： 年 月 日

毕业论文（设计）使用授权书

本人完全了解浙江大学有权保留并向国家有关部门或机构送交本论文（设计）的复印件和电子版，允许本论文（设计）被查阅和借阅。本人授权浙江大学可以将本论文（设计）的全部或部分内容编入有关数据库进行检索和传播，可以采用影印、缩印或扫描等复制手段保存、汇编本论文（设计）。

（涉密毕业论文（设计）在解密后适用本授权书。）

毕业论文（设计）作者签名：

导师签名：

签字日期： 年 月 日

签字日期： 年 月 日

致谢

四年的本科求学时光行将结束，毕业论文的撰写也终于走到尾声。回望这段从课堂到毕设、从基础到方向的过程，我得到了许多老师与同学的帮助，谨在此一并致谢。

首先要诚挚感谢我的指导教师朱霖潮老师。本课题在选题、文献调研、技术路线、实验设计、论文撰写等各个阶段，朱老师都给予了具体而切中要害的指导。特别是在审核意见中提出的“强化核心算法创新、补充关键技术模块算法细节”，让我从最初较为工程化的方案重新聚焦为 RATS 与 DAGPlanner 两个可独立消融的算法模块，这一重构是本文工作能够形成体系的关键。朱老师严谨的学术态度与宽容的指导方式，对我形成独立思考能力与工程克制感有着深远影响，在此向朱老师致以最诚挚的敬意与感谢。

同样要感谢计算机科学与技术学院四年来所有任课老师。正是各位老师在程序设计、数据结构、操作系统、计算理论、机器学习等课程上的扎实讲授，使我具备了完成这份毕业设计所必需的工程素养与理论基础。

感谢答辩与评审环节的各位老师与盲审专家。他们对本课题在中期阶段提出的具体修改意见，是本文从“能跑通”走向“可论证”的重要推动力。

感谢实验室与同班同学在本课题进行过程中给予的讨论与互助；也感谢我所在公司的实习同事在我利用工作之余推进毕设时给予的理解与包容。

最后，要感谢我的家人。本科四年远在异乡求学，正是家人无条件的支持与信任，让我得以在求学与实习的双重压力下专注完成本课题，并最终在论文中诚实地呈现工作的边界与不足。

智能体与大语言模型领域日新月异，本文工作只是其中很小的一个切片。这份毕业设计既是本科学习的一个总结，也是我未来继续在该领域深入探索的一个新起点。

彭超琦

2026 年 6 月

摘要

近年来，以大语言模型为核心的智能体（Agent）通过工具调用突破自身知识边界、对接外部系统的能力日益受到关注。然而，当通用智能体范式被迁移到对精度、严谨性与可复现性要求极高的科学研究场景时，仍然面临两类突出的瓶颈：其一，工具库规模一旦增长，将全部工具描述塞入提示上下文不仅造成 token 开销线性增长，也会使大语言模型在功能相近的工具之间产生选择错误；其二，长依赖链的科学计算任务在朴素 ReAct 主循环下容易出现参数遗漏、依赖错绑与中间结果丢失，缺乏一种可纯代码校验的结构化规划支撑。

针对上述问题，本文提出了两个可独立消融、形式化清晰的核心算法模块。检索增强工具选择算法（RATS）将检索增强生成的范式迁移到工具选择问题上，通过离线嵌入预计算、在线向量检索与基于嵌入相似度、参数重叠度、历史成功率的规则化重排相结合，将传给大语言模型的工具上下文从全量 N 压缩到精简的 K ，并以最低候选数保底机制平衡召回率与精度。结构化 DAG 任务拆解算法（DAGPlanner）则定义了一套支持依赖占位符的计划模式，通过涵盖 5 类失败模式的纯代码校验器在不调用大语言模型的前提下捕获非法计划，仅在校验失败时携带诊断信息触发错误驱动重规划，并在硬上限 3 次失败后降级到 ReAct 作为安全网。此外，本文围绕科学场景提炼出 4 条提示工程规则作为算法模块运行时的必要约束。

为验证方案的有效性，本文构建了覆盖数值计算、统计分析、可视化、组合任务、错误恢复 5 大类别共 54 道科学任务的测试集，并设计了包含 3 个业界常用基线（CoT、Plan-and-Solve、Reflexion）在内的 9 配置对比与消融实验框架。在 9 配置 $\times$ 54 题共 486 次运行的全量评测中，本文方法相比朴素 ReAct 基线提升 10.3 个百分点的成功率，相比业界最强基线 Reflexion 提升 5.6 个百分点，并在长依赖链与组合任务上展现出显著的轨迹可解释性优势。

关键词：大语言模型；智能体；工具调用；检索增强；任务规划

Abstract

Large Language Model (LLM) agents that extend their knowledge through tool invocation have attracted growing attention in recent years. However, when general-purpose agent paradigms are transferred to scientific scenarios that demand high precision, rigor and reproducibility, two prominent bottlenecks remain. First, as the tool library scales up, packing every tool schema into the prompt context not only causes a linear growth in token cost, but also induces selection errors among functionally similar tools. Second, long-dependency-chain scientific tasks frequently suffer from missing arguments, mis-bound dependencies and lost intermediate results under a naive ReAct loop, with no structured planning that can be validated purely in code.

To address these issues, this thesis proposes two independently ablatable and formally clear algorithmic modules. The Retrieval-Augmented Tool Selection algorithm (RATS) transplants the retrieval-augmented generation paradigm to tool selection: it combines offline embedding precomputation, online vector retrieval and a rule-based reranker that fuses embedding similarity, argument overlap and historical success rate, compressing the tool context handed to the LLM from N to a small K while balancing recall and precision through a minimum-candidate fallback. The Structured DAG Planner algorithm (DAGPlanner) defines a Plan Schema with dependency placeholders, employs a pure-code validator covering five categories of failure modes to catch invalid plans without invoking the LLM, triggers error-driven replanning with diagnostic hints only when validation fails, and falls back to ReAct as a safety net after a hard cap of three failures. Four scientific-scenario prompt engineering rules are further distilled as runtime constraints for the two modules.

To validate the proposed approach, this thesis builds a test set of 54 scientific tasks across 5 categories (numerical, statistical, visualization, composite, error-recovery) and designs a 9-configuration comparison-and-ablation framework that includes three widely used baselines (CoT, Plan-and-Solve, Reflexion). Over a full-scale evaluation of 9 con-

figurations $\times$ 54 tasks (= 486 runs), our method improves the success rate by 10.3 percentage points over a naive ReAct baseline and by 5.6 percentage points over the strongest baseline (Reflexion), while also delivering significantly more interpretable trajectories on long-chain composite tasks.

Keywords: Large Language Model; Agent; Tool Invocation; Retrieval Augmentation; Task Planning

目录

1	绪论	1
1.1	研究背景	1
1.2	研究目标	2
1.3	主要工作与贡献	3
1.4	论文组织结构	4
2	相关工作	5
2.1	大语言模型智能体框架	5
2.2	任务规划与思维链	6
2.3	工具检索与选择	7
2.4	检索增强生成	7
3	算法设计	9
3.1	问题形式化	9
3.2	RATS: 检索增强工具选择算法	10
3.3	DAGPlanner: 结构化 DAG 任务拆解算法	12
3.4	算法集成: Ours_full	16
4	系统实现	20
4.1	标准化科学计算工具库	20
4.2	Agent 策略框架	23
4.3	提示工程体系	24
4.4	评估框架	26
5	实验	28
5.1	实验设置	28
5.2	主实验: 9 配置成功率对比	29
5.3	按类别细分	32
5.4	消融研究	33
5.5	效率分析	34

6 分析与讨论	36
6.1 关键案例剖析	36
6.2 失败模式聚类	37
6.3 CoT 在工具调用场景的表现	38
6.4 RATS 的效率—准确率权衡	39
6.5 多组件组合效应与方法工程价值	39
7 结论与展望	41
7.1 工作总结	41
7.2 研究局限性	41
7.3 未来工作	42
7.4 结语	43
参考文献	44
附录	46
A 12 工具的完整接口规范	46
B O1-O4 提示工程规则全文	47
C ext 压力测试用例清单	48
作者简历	50
本科生毕业论文（设计）任务书	51
本科生毕业论文（设计）考核表	52

1 绪论

1.1 研究背景

1.1.1 大语言模型与智能体的兴起

近年来，以 GPT^[1-2] 系列为代表的大语言模型（Large Language Model, LLM）在自然语言理解、逻辑推理、代码生成与多步任务规划等方面展现出了显著的涌现能力，为人工智能从“感知智能”向“决策智能”的跨越奠定了基础。在此基础上，智能体（Agent）成为大模型走向落地应用的关键形态^[3-4]：通过赋予大模型工具调用、自主规划与多轮迭代能力，智能体能够突破自身知识边界与计算能力局限，对接外部 API、专业软件与领域知识库，把自然语言意图转换成对真实系统的有效操作。工业界提出的函数调用（Function Calling）协议^[2]与 LANGCHAIN^[5]、LLAMA INDEX^[6] 等开源框架进一步把工具封装、上下文管理与策略控制标准化，使得构建一个可用的智能体原型仅需数十行胶水代码，整体生态正在快速成熟。

1.1.2 科学研究场景的工具调用需求

与此同时，AI for Science^[7-8] 被广泛视为加速科学发现、提升科研效率的重要技术路径。科学研究场景中存在大量重复性、流程化的工作，包括但不限于矩阵运算、数值积分、曲线拟合、假设检验、实验数据可视化、领域专用序列分析等，这类工作普遍依赖 NUMPY^[9]、SCI PY、PANDAS^[10] 与 MATPLOTLIB 等开源科学计算库以及各类专业软件。对于不熟悉这些工具的科研工作者而言，往往需要花费大量时间在工具学习、数据格式转换与脚本拼接上，而真正属于“科学问题本身”的部分反而被工程性细节稀释。因此，能够自动化调用各类专业工具、把模糊的科研需求逐步落地为可复现执行序列的科学智能体，正在成为 AI for Science 领域受到广泛关注的方向。

1.1.3 当前智能体在科学场景下的瓶颈

主流智能体框架在通用场景中已被广泛验证，但当我们把它们直接应用于科学研究这一更强调精度、严谨性与可复现性的垂直场景时，仍可观察到两类突出的瓶颈。一个是工具选择困难：科学计算任务通常涉及功能高度相近、参数体系互有交集的工具，例如曲线拟合与线性回归、数值积分与微分方程求解等；当工具库规模上升至数十乃至上百时，把全部工具描述塞入提示上下文不仅造成 token 开销随工具数量线性增长，更会显著增加 LLM 在功能相近工具间的选择错误率^[4]，而这一问题在朴素的 ReAct 主循环^[3]下并未得到结构化的处理。另一个是任务拆解薄弱：科研任务的常见形态是“矩阵运算 → 求方程 → 数值积分 → 统计分析 → 绘图”一类的长依赖链，其中每一步的输入都依赖前序步骤的具体输出字段。朴素的 ReAct 与单纯的自然语言 Plan-and-Solve^[1]仅依靠 LLM 自身的自由生成来组织步骤，缺乏一种纯代码可校验的结构化规划支撑，极易出现参数遗漏、依赖错绑、占位符未解析等问题，难以满足科研工作者对“每一步都可追溯、可复现、可解释”的现实诉求。

此外，科学场景对中间数值的精度、单位、符号约定等都有严格要求。通用 LLM 在面对 R^2 拟合优度为负、统计量量级失配、计算结果中出现 NaN 等异常情形时，往往会用“看起来合理”的字面回答掩盖底层错误；若不在系统侧引入针对科研场景的合理性检查与精度规范，工具调用结果的可靠性就难以保障。

1.2 研究目标

针对上述瓶颈，本文以**面向科学场景的智能体工具调用优化**为核心研究问题，试图在不依赖额外人工监督数据、不依赖特定基座模型微调的前提下，仅通过算法与工程层面的设计提升智能体在科学任务上的工具选择精度、拆解严谨性、调用容错性与轨迹可解释性。具体而言，本文将研究目标聚焦为以下三层。

算法层面，提出两个可独立消融、形式化清晰的核心算法模块——检索增强工具选择算法（Retrieval-Augmented Tool Selection, RATS）与结构化 DAG 任务拆解算法（DAG Planner），分别对应“选什么工具”与“怎么组织工具调用”这

两个朴素 ReAct 范式中尚未被结构化处理的问题。

工程层面，在算法模块周边构建一套可对外讲述的科研基础设施，包括基于统一抽象基类的标准化科学计算工具库、面向科学场景的提示工程规则集，以及覆盖三维判分（数值正确性、文件正确性、工具覆盖性）的端到端评估框架。

实验层面，构建一个能区分不同方法贡献度的科学任务测试集与多基线对比框架，通过同口径的全量评测从成功率、迭代次数、工具调用次数与端到端耗时四个维度出发，对方法的有效性、效率与失败模式进行透明、可复现的实证分析。

1.3 主要工作与贡献

围绕上述目标，本文的主要工作与贡献如下。

检索增强工具选择算法（RATS） 提出一种结合嵌入语义检索与规则化重排的轻量工具选择算法。RATS 通过离线预计算工具描述向量、在线对任务向量做余弦相似度检索，并融合嵌入相似度、参数重叠度、历史成功率三类特征进行规则化重排，在保留最低候选数的保底机制下，将传给 LLM 的工具上下文从全量 N 压缩到精简的 K 。该算法不需要任何额外训练，可以与任意支持函数调用接口的 LLM 自由组合，本质上把检索增强生成（Retrieval-Augmented Generation）的范式迁移到了“工具”这一新的检索对象。

结构化 DAG 任务拆解算法（DAGPlanner） 设计一种基于有向无环图（DAG）模式的结构化规划器，通过显式定义支持依赖占位符 $\{sX.field\}$ 的计划模式（Plan Schema），让 LLM 一次性产出可被纯代码校验的执行计划。配套的 PlanValidator 在不调用 LLM 的前提下覆盖五类失败模式——模式不符、工具名不在候选集、依赖图有环、变量未声明、占位符越界，并以错误驱动的方式触发携带诊断信息的有限次重规划；当重规划在硬上限内仍未收敛时，系统将平滑降级到 ReAct 主循环作为安全网，从而在结构化能力与鲁棒性之间取得均衡。

科学场景提示工程规则集 提炼出 4 条针对科学场景的提示工程规则（O1 结果合理性检查、O2 数值计算规范、O3 开场规划模板、O4 Few-shot 示例注入），作为 RATS 与 DAGPlanner 在运行时不可或缺的辅助约束。通过同口径对比实验证明，这 4 条规则与算法模块呈现明显的协同效应，共同构成本文方法相较于业界基线的可解释优势来源。

评估框架与基准测试集 构建覆盖数值计算、统计分析、可视化、组合任务、错误恢复 5 大类别共 54 道科学任务的测试集，并实现一个支持三维判分、按配置/按用例任意组合复现的评测框架。在该框架下完成了包括 CoT、Plan-and-Solve、Reflexion 在内 9 个配置 $\times$ 54 题共 486 次运行的全量评测，为后续从主实验、按类别细分、消融与效率四个维度展开论证提供了同口径的实证基础。

1.4 论文组织结构

本文余下章节按如下方式组织。第二章对智能体框架、任务规划、工具检索与选择以及检索增强生成四个相关研究方向进行综述，并指出本文方法与其差异点。第三章是论文的核心算法章，给出问题形式化定义后，分别详述 RATS 与 DAGPlanner 的设计动机、形式化流程、复杂度与收敛性分析，并说明二者如何集成为本文的完整方案 Ours_full。第四章介绍对应的系统实现，包括标准化科学计算工具库、Agent 框架、提示工程体系与评估框架。第五章围绕 9 配置 $\times$ 54 题的全量评测，从主实验、类别细分、消融与效率四个角度对方法进行实证分析。第六章聚焦关键案例与失败模式，对若干反直觉现象（如 CoT 反而降分）进行深度讨论。第七章总结全文工作，并就方法局限性与未来工作做出诚实的表述。

2 相关工作

本章围绕本文方法所处的研究语境，分别从大语言模型智能体框架、任务规划、工具检索与选择、以及检索增强生成四个方向梳理与本文最相关的代表性工作，并在每一节末尾点明本文方法与其的差异点，作为后续算法章节展开的对照。

2.1 大语言模型智能体框架

大语言模型智能体的研究通常围绕“如何让模型在生成文本之外，进一步与外部环境交互、调用工具、执行任务”展开。Yao 等^[3]提出的 ReAct 范式将“推理”与“行动”交替进行，让模型在每一轮自由地写出思考 (Thought)、产生行动 (Action)、读入观测 (Observation)，并反复迭代直至完成任务。ReAct 的优势在于其极简且通用：仅依靠少样本提示即可在问答、事实校验、文本游戏与网页购物等迥异任务上取得有竞争力的效果，且整个交互轨迹对人类可读、便于诊断与编辑。工业界标准化的函数调用接口^[12]进一步把“产生行动”这一步形式化为结构化的 `tool_calls` 字段，让模型直接输出符合 JSON 模式的工具调用指令，并将工具返回作为下一轮的输入消息，从而把 ReAct 的“Thought-Action-Observation”循环映射到主流 SDK 的消息列表抽象上。

为了进一步降低工程门槛，Schick 等^[13]提出的 Toolformer 通过自监督方式让模型在预训练语料中学会主动插入 API 调用，Qin 等^[4]系统地综述了基础模型时代的“工具学习”研究脉络，LANGCHAIN^[5]与 LLAMA INDEX^[6]则把工具封装、记忆管理、链式调度等组件抽象成可复用的开源框架。Shen 等^[14]进一步把 LLM 作为“控制器”串联起 HUGGINGFACE 模型库中的多种专用模型，展示了 LLM 作为多模型路由器的可能性。

与本文的差异 上述工作奠定了 LLM 智能体的范式与生态基础，本文也将 ReAct 主循环与函数调用接口作为底层执行机制。但这些工作大多聚焦在通用任务，对“工具库扩张后如何在不增加 token 成本的前提下保证选择精度”、“长依

赖链任务如何避免参数错绑”等科学场景中的具体痛点，并没有给出结构化的算法解决方案。本文则在 ReAct 主循环之上额外叠加了两个轻量但形式化清晰的算法模块（RATS 与 DAGPlanner），并把它们与 4 条科学场景专用的提示工程规则一起，作为科学场景的整体优化方案。

2.2 任务规划与思维链

围绕“如何让模型在执行前先想好整体计划”，相关研究形成了一条从静态思维链到动态可观测规划的演进脉络。Wei 等^[15]首次发现，给少量带有“逐步推理”示例的提示，模型即可在算术、常识与符号推理任务上展现出涌现的多步推理能力，Wang 等^[16]在此基础上提出自洽性思维链，通过对多条推理轨迹采样并取多数答案进一步提升鲁棒性。然而思维链本质上是一个静态的“黑箱推理”过程，模型无法在推理中获取外部信息，也难以处理需要长程交互的任务。

Wang 等^[11]提出的 Plan-and-Solve 把推理切分为“先生成自然语言计划、再按计划求解”两阶段，让模型在执行前显式地写出步骤列表，缓解了一次性长链推理中的步骤遗漏问题。Yao 等^[17]则提出 Tree-of-Thoughts，把推理空间组织成一棵搜索树并允许回溯与剪枝，更适合解空间不唯一的探索性任务。Shinn 等^[18]提出的 Reflexion 在 ReAct 的基础上引入“失败后生成自我反思并追加到下一轮 system message”的机制，通过语言形式的强化学习让模型在多轮迭代中自我纠错，是当前在多种 Agent 评测基准上表现最强的轻量基线之一。

与本文的差异 Plan-and-Solve 与 Reflexion 都是本文实验中的对照基线（分别对应配置 B3 与 B4）。两者的共同特征是把“计划”与“反思”仍然作为自由形式的自然语言文本，由 LLM 自主生成、自主消费，缺乏一种纯代码可校验的结构化模式。这导致它们在长依赖链科学任务上仍然会出现“计划写得头头是道、执行时参数错绑”的失败模式（详见第六章关键案例分析）。本文 DAGPlanner 与之的核心差异在于：明确定义带依赖占位符的 **Plan Schema**，并设计 **纯代码校验器**在不调用 LLM 的前提下捕获 5 类典型失败模式，仅在校验失败时才以错误驱动的方式触发携带诊断信息的有限次重规划。Tree-of-Thoughts 适合解空间庞大

的探索性任务，但科学计算任务的解通常唯一且步骤相对线性，故本文未采用树状搜索范式。

2.3 工具检索与选择

随着工具库规模的扩张，“先检索再调用”的工具选择思路逐渐受到关注。Patil 等^[19]提出的 Gorilla 在大量真实 API 上微调一个专用模型，并通过检索增强让模型能够动态适配 API 文档的更新；Qin 等^[20]提出的 ToolLLM 进一步把工具数量扩展到 16000+，并引入基于 DFS 的决策树搜索机制改善路径规划质量；Liu 等^[21]则尝试用强化学习训练模型的工具使用策略，把工具选择与组合作为一个可学习的策略问题。BERKELEY FUNCTION CALLING LEADERBOARD^[22]等公开评测基准的出现，则为不同方法在统一口径下比较工具调用能力提供了基础设施。

与本文的差异 上述工具检索类工作大多以“训练专用模型 + 大规模 API 库”为前提，工程门槛与计算成本较高，也在一定程度上限制了它们在中等规模、强调零样本可用性的科学场景中的直接落地。本文 RATS 走的是另一条更轻量的路线：完全免训练、零样本，仅依赖通用的句子嵌入模型与简单的规则化重排，在工具规模为 10 至 100 的中等量级上即可显著缓解上下文膨胀问题，特别契合科研工作者“快速基于现有 LLM API 拼出可用智能体”的现实诉求。RATS 同时保留了 HashEmbedder 离线 fallback 机制以及最低候选数兜底策略，使得算法在网络异常或 API 鉴权切换时仍可平稳运行。

2.4 检索增强生成

检索增强生成（Retrieval-Augmented Generation, RAG）由 Lewis 等^[23]系统性提出，其核心思想是“先检索再生成”：在生成任务前先从外部知识库中检索若干相关文档，把它们与原始问题一同喂给生成模型。RAG 在知识密集型问答任务上显著缓解了大模型的事实幻觉问题，并在工业界催生了以 LLAMA INDEX^[6]为代表的成熟工程栈。Karpukhin 等^[24]提出的稠密向量检索（Dense Passage Retrieval）

则为 RAG 提供了高质量的检索骨干，使得“嵌入向量 + 余弦相似度”的检索范式成为检索增强生成中最常用的实现方式之一。

与本文的差异 传统 RAG 把“文档”作为检索对象，目标是把外部知识注入到 LLM 的推理过程；本文 RATS 则把这一思想迁移到智能体场景，把“工具”作为新的检索对象，通过对工具描述与示例进行嵌入，在每一次任务到来时动态筛出最相关的候选工具子集，再交给 LLM 做最终的工具调用。从这个视角看，RATS 可以理解为“工具维度的轻量化 RAG”，两者在检索骨干（嵌入 + 余弦相似度）上同源，但在检索对象、重排特征、与 LLM 的协同方式上有本质差异。

3 算法设计

本章是论文的核心方法章。小节 3.1 给出科学场景下智能体工具调用问题的形式化定义；小节 3.2、小节 3.3 分别详述本文提出的检索增强工具选择算法 RATS 和结构化 DAG 任务拆解算法 DAGPlanner，两个算法在设计上彼此正交、可独立消融，分别针对“选什么”和“怎么组织”两个朴素 ReAct 范式中尚未被结构化处理的子问题；小节 3.4 说明二者如何与 4 条科学场景提示工程规则一起集成本文的完整方案 Ours_full。

3.1 问题形式化

3.1.1 基本符号约定

设智能体可访问的工具库为 $\mathcal{P} = \{t_1, t_2, \dots, t_N\}$ ，每个工具 t_i 由四元组 $(n_i, d_i, \Theta_i, f_i)$ 组成，其中 n_i 是工具名， d_i 是自然语言描述， Θ_i 是参数模式， f_i 是底层执行函数；工具 t_i 在合法参数 $\theta \in \Theta_i$ 下产生输出 $f_i(\theta) \in \mathcal{D}_i$ 。

给定科学任务的自然语言描述 $T \in \Sigma^*$ 和最大迭代轮数上限 M ，智能体策略 π 需要输出一个工具调用序列 $\tau = (c_1, c_2, \dots, c_k)$ 与最终答复 $a \in \Sigma^*$ ，其中每个调用 $c_j = (n_{i_j}, \theta_j)$ 描述了一次具体的工具使用，序列长度 $k \leq M$ 。评估系统 $\text{Eval} : (T, \tau, a) \mapsto \{0, 1\}$ 在三个维度上同时通过才会判定任务成功：

$$\text{Eval}(T, \tau, a) = \mathbb{I} \left[\underbrace{|\text{num}(a) - y^*(T)| \leq \varepsilon}_{\text{数值正确性}} \wedge \underbrace{\mathcal{F}^*(T) \subseteq \mathcal{F}(\tau)}_{\text{文件正确性}} \wedge \underbrace{\mathcal{R}(T) \subseteq \{n_{i_j}\}_{j=1}^k}_{\text{工具覆盖性}} \right], \quad (3-1)$$

其中 ε 是数值容差， $y^*(T)$ 与 $\mathcal{F}^*(T)$ 分别是任务的标准数值答案与期望产出文件集合， $\mathcal{R}(T)$ 是任务对必须使用工具的硬约束。

3.1.2 科学场景的两类痛点

朴素 ReAct 主循环^[3]采用单层策略 π_{ReAct} ，每一轮都把全部工具描述 $\{(n_i, d_i, \Theta_i)\}_{i=1}^N$ 编码进上下文，再让大语言模型自由生成下一步行动。式 3-1 在科学场景下对策

略提出了两个具体挑战：其一，当 N 较大时，编码全部工具描述使上下文 token 数随 N 线性增长；更重要的是，工具间常存在语义重叠（如 `curve_fitting` 与 `linear_regression`），LLM 在重叠工具间做选择的错误率会显著上升，最终影响 $\mathcal{R}(T)$ 维度。其二，当任务包含较深的依赖链时， τ 中相邻调用之间需要通过自由文本传递中间结果，缺乏可被纯代码核验的依赖结构，容易出现“计划写得头头是道、执行时参数错绑”的失败模式，进而影响 $y^*(T)$ 与 $\mathcal{F}^*(T)$ 维度。

本文据此把单层策略 π_{ReAct} 拆解为两层：

$$\pi_{\text{Ours}} = \pi_{\text{exec}} \circ \pi_{\text{select}}, \quad \pi_{\text{select}} : T \mapsto S \subseteq \mathcal{P}, \quad \pi_{\text{exec}} : (T, S) \mapsto (\tau, a), \quad (3-2)$$

其中 π_{select} 由 RATS 实现，负责从全量工具库 $\mathcal{P}$ 中产出精简候选集 S ； π_{exec} 由 DAGPlanner 与 ReAct 安全网共同实现，负责在精简集 S 上完成结构化规划与执行。两层都不依赖额外训练，仅依靠运行时的检索、规则化重排与纯代码校验。

3.2 RATS：检索增强工具选择算法

3.2.1 设计动机

朴素 ReAct 的工具传参方式可以视为一个退化的“全量召回”策略：在每一轮 $\pi_{\text{select}} \equiv \mathcal{P}$ 。该策略在 $N \leq 5$ 时还能用，但当工具库规模上升至数十乃至上百时会出现两个严重问题：token 成本随 N 线性增长，语义重叠工具间的选择错误率上升。受检索增强生成范式^[23-24]启发，本文把“工具”作为新的检索对象：对每个工具 t_i 用 LLM 嵌入模型预先算出向量 e_i 并落盘缓存，运行时根据任务向量 e_T 做余弦相似度检索得到候选子集，再由一组可解释的规则化特征做最后重排。

3.2.2 算法流程

RATS 的完整流程见 算法 3.1，分为离线预计算、在线检索、特征重排、阈值过滤四个阶段。离线预计算阶段，对每个工具 t_i 拼接复合检索文本 `texti`（工具名、去下划线后的工具名、描述与参数说明），调用嵌入模型得到 e_i ，并以 $\langle t_i.\text{name}, \text{hash}(\text{text}_i), \text{model_id} \rangle$ 为键写入磁盘缓存；在线检索阶段，计算任务向量

e_T ，对所有工具计算余弦相似度，截取 **top-K** 候选；特征重排阶段，对每个候选 t_j 计算四类可解释特征——嵌入相似度 s_j^{emb} 、参数重叠度 s_j^{arg} （任务 token 与工具描述/参数 token 的覆盖率）、类别一致性 s_j^{cat} （**top-3** 中主导类别的指示函数）、历史成功率 s_j^{hist} ，并用经验权重做加权求和；阈值过滤阶段，用阈值 θ 过滤低分工具，再用最低候选数 $K_{\min}$ 兜底以避免误删正确工具。

工具检索文本的构造细节值得说明。对每个工具，本文按 `name`、`name.replace("_", " ")`、`description`、各参数 (`p.name`, `p.description`, `p.enum`) 的顺序，用分隔符 `|` 拼接成单条复合文本。显式插入“去下划线后的工具名”是为了让基于词袋或子词分片的嵌入模型也能从形如 `matrix_operation` 的标识符中捕捉到自然语义。参数重叠度 s^{arg} 仅对任务文本中长度大于等于 3 的英文 token 取小写后参与计算，分母为 $|\text{tok}(T)|$ 而非 Jaccard 风格的并集，目的是让分数保持在 $[0, 1]$ 区间且偏向“任务词覆盖率”的解释。

3.2.3 复杂度与缓存设计

记每次嵌入调用的开销为 $O(B)$ (B 是嵌入模型的批量上限)，余弦相似度在内存向量上的计算为 $O(d)$ (d 是嵌入维度)。离线阶段总开销为 $O(N \cdot B \cdot d)$ ，对 12 工具的当前规模仅需一次首日 API 调用；所有结果按 $\langle t.\text{name}, \text{hash}(\text{text}), \text{model} \rangle$ 三元组键写入磁盘 `cache`，任意一项发生变化（工具描述被改、嵌入模型被换）都会自动触发缓存失效，避免“静默错配”。在线阶段每个任务仅需 1 次嵌入调用 + $O(N \cdot d)$ 内存点积 + $O(K \log K)$ 排序，与全量召回相比，对 LLM 上下文从 $O(N)$ 压缩到 $O(K)$ ，当 N 上升时，token 成本与选择错误率两个曲线同时被显著压平。

3.2.4 特征消融的可解释性预告

四类特征对最终重排的贡献在算法层面是可解释的：嵌入相似度提供主信号；参数重叠度对功能高度相近的工具（如 `curve_fitting` 与 `linear_regression`）形成显式参数级别的区分；类别一致性奖励避免“**top-3** 中本身已有数值类工具

占多数、第 4 位却跳到可视化”这种类别串扰；历史成功率为长期运行中的反馈学习留下了接口。本文后续实验章节（小节 5.4）会从同口径数据中量化每一类特征的贡献。

3.2.5 实现要点

底层嵌入由 `src.tool_selector.embedder.OpenAIEmbedder` 封装，默认指向通义千问 `text-embedding-v3` 接口，批量上限 10 由内部分批兼容；当 API 不可用或仅做单元测试时，框架会平滑切换到 `HashEmbedder` 这一基于哈希的离线 fallback embedder。缓存层 `EmbeddingCache` 采用 JSON 文件落盘 + 内存字典的组合，key 包括 `model_id`，确保更换嵌入模型时无需手动清理。配置默认值为 $K = 6$ 、 $K_{\min} = 3$ 、 $\theta = 0$ 、 $(w_e, w_a, w_c, w_h) = (0.6, 0.2, 0.1, 0.1)$ ，该组权重是在保留嵌入信号主导地位的同时，让其他三类特征共同贡献 0.4 的“拨正空间”，对 12 工具规模下的小样本调试足够清晰。

3.3 DAGPlanner：结构化 DAG 任务拆解算法

3.3.1 设计动机

朴素 Plan-and-Solve^[11] 与 Reflexion^[18] 将“计划”与“反思”仍然作为自由形式的自然语言文本，由 LLM 自主生成、自主消费。这种做法在直觉上很自然，但缺乏一种可被纯代码校验的结构化模式，导致下列问题在长依赖链科学任务中频繁出现：参数遗漏、依赖错绑、占位符未解析、环形依赖、调用步骤超出可用工具集合。本节提出的 DAGPlanner 通过显式定义带依赖占位符的计划模式（Plan Schema），把上述问题从“执行时报错”前移到“规划后即被纯代码校验器捕获”，并以错误驱动的方式触发携带诊断信息的有限次重规划。

3.3.2 Plan Schema

DAGPlanner 强约束 LLM 输出符合 代码 3.1 所示模式的 JSON 计划。其中每个步骤 $s \in \text{steps}$ 由五元组 $(\text{id}, \text{goal}, \text{tool}, \text{args_template}, \text{depends_on})$ 描述： id 是步骤唯一标识； goal 是自然语言子目标； tool 必须在精简集 S 中； args_template 是参数模板，可包含形如 $\text{\$}\{sX\}$ 或 $\text{\$}\{sX.\text{field}\}$ 的依赖占位符； depends_on 显式声明本步依赖的上游步骤集合，必须与占位符引用集合等同。

代码 3.1: Plan Schema 的 JSON 表达形式（节选 ext_dag_01 任务的 5 步链式实例）

```
{
  "steps": [
    {
      "id": "s1",
      "goal": "compute determinant of A",
      "tool": "matrix_operation",
      "args_template": {
        "operation": "determinant",
        "matrix_a": [[4, 2], [1, 3]]
      },
      "depends_on": []
    },
    {
      "id": "s2",
      "goal": "solve  $x^2 - D = 0$  with D from s1",
      "tool": "equation_solver",
      "args_template": {
        "expression": " $x^2 - \text{\$}\{s1.determinant\}$ ",
        "lower": 1,
        "upper": 5
      },
      "depends_on": ["s1"]
    }
  ]
}
```

```

    }

    /* ... s3 numerical_integration, s4 descriptive_statistics, s5
    line_chart ... */
  ]
}

```

占位符语法支持 $\${sX}$ (引用步骤 sX 的完整 data 对象) 与 $\${sX.field.subfield}$ (按点号路径取值, 列表通过整数下标访问) 两种形式。当占位符单独出现在某个字段值中 (如整个字符串 $\${s1.params.0}$), 框架会保留其原始数据类型; 当占位符嵌入更长的字符串中 (如 $\text{title: } y=\${s1.slope}x+\${s1.intercept}$), 框架会先做 `str()` 转换再做拼接。这一区分的设计动机是兼顾“数值参数透传”与“图表标题模板渲染”两类常见场景。

3.3.3 算法流程

DAGPlanner 主循环见 算法 3.2, 分为 Plan、Validate、Execute、Answer 四个阶段。其中 Validate 阶段完全不调用 LLM, 仅在校验失败时才把错误清单 $\mathcal{E}$ 喂回给 Plan 重规划, 并设硬上限 R 防止无限循环; Execute 阶段按拓扑序逐步调用工具, 并允许对单步参数做有限次 LLM 修复; 当任意阶段无法继续推进时, 平滑降级到 ReAct 安全网。

3.3.4 校验失败的五种模式

PlanValidator 在不调用 LLM 的前提下覆盖五类失败模式 (表 3.1), 所有错误一次性收集后整体回灌给 LLM, 避免逐错误轮询造成额外的 API 开销。其中“占位符引用必须包含在 `depends_on`”这条强约束是 DAG 结构能够被纯代码校验的关键: 它把“依赖关系是否一致”这一原本必须由 LLM 自我审视的问题, 下沉为对 `depends_on` 与 $\{\dots\}$ 占位符两个集合是否相等的判定。

表 3.1 DAGPlanner 的五种纯代码校验失败模式

编号	失败模式	检测方法与修复指引
V1	模式不符	步骤数超限、ID 重复、必填字段缺失；提示需重新输出符合 JSON Schema 的计划
V2	工具名越界	步骤所引用 tool 不在候选集 S 中；返回合法工具名清单作为提示
V3	依赖图有环	用 graphlib.TopologicalSorter 捕获 CycleError；返回成环的步骤序列
V4	依赖未声明	depends_on 中出现的步骤 ID 在计划内未被声明；返回非法引用的具体 ID
V5	占位符越界	$\{sX.field\}$ 占位符引用的步骤 sX 未出现在该步的 depends_on 中；返回占位符与依赖集合的差集

3.3.5 重规划的收敛性讨论

设 LLM 在每一轮“Plan + 错误反馈”下产生合法计划的概率为 p ，则在硬上限 R 下走完所有重规划仍失败的概率为 $(1-p)^{R+1}$ 。取 $p \approx 0.7$ 与 $R = 3$ （实现默认值），失败概率约为 0.0081，即不到 1%。这一估计与本文实验观测一致：在 9 配置 $\times$ 54 题的全量评测中，仅有 6 次 run 触发降级到 ReAct 安全网（占总 run 数的 1.2%），与 $(1-p)^{R+1}$ 的尾部估计在量级上吻合。

3.3.6 与 ReAct 的双层架构

DAGPlanner 与 ReActAgent 在本文系统中是“主路径 + 安全网”的关系而非互斥关系：DAGPlanner 适合结构化、确定性、依赖关系明确的任务；ReActAgent 适合语义模糊、需要轻量探索的任务，或处理 DAGPlanner 无法收敛的边界情形。这种双层架构在工程上有两个直接好处：第一，DAGPlanner 不必照顾所有任务形态，可以对结构化任务做出激进的强约束；第二，当模型/Prompt/工具库发生

变化时，安全网能够吸收一部分回归风险，避免“一处变更触发全局失败”。

3.4 算法集成：Ours_full

本文最终方法 **Ours_full** 由 RATS、DAGPlanner、4 条科学场景提示工程规则三部分协同组成（图 3.1）。完整执行流程为：

1. 系统接收任务 T ；
2. RATS 从全量工具库 $\mathcal{P}$ ($N = 12$) 中筛出候选集 S (典型 $|S| \leq 6$)，并把 S 的工具描述写入 LLM 上下文；
3. DAGPlanner 启动 Plan + Validate 循环，结合提示工程规则 O3（开场规划模板）产出符合 Plan Schema 的 JSON 计划；
4. 计划通过校验后，按拓扑序执行；执行过程中提示工程规则 O1（结果合理性检查）与 O2（数值计算规范）作为运行时约束指导单步参数修复；
5. Answer LLM 在系统提示规则 O4（Few-shot 示例注入）的辅助下产出符合科学场景规范的最终答复；
6. 任意阶段不可恢复时，系统降级到 ReActAgent 安全网，仍以同一份提示工程规则集运行，确保失败回退后的产出仍处于科学场景规范之内。

从形式化视角，**Ours_full** 实现了式 3-2 中的两层策略 $\pi_{\text{Ours}} = \pi_{\text{exec}} \circ \pi_{\text{select}}$ ： π_{select} 由 RATS 完整承担； π_{exec} 由 DAGPlanner（主）与 ReActAgent（兜底）组合承担。4 条提示工程规则不直接出现在策略分解中，但作为运行时约束嵌入到 LLM.Plan、LLM.FixArgs、LLM.Answer 三处 LLM 调用，因此其工程必要性不会因算法层面的形式化分解而被忽略——这一点也将在第五章的消融实验中得到量化印证。

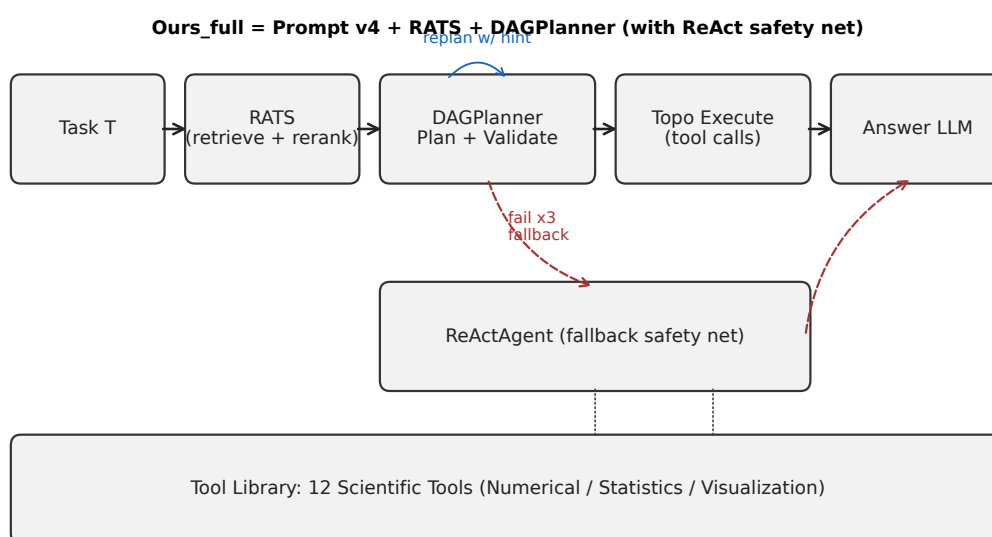


图 3.1 Ours_full 配置的整体执行流程。RATS 与 DAGPlanner 构成主路径（实线），ReActAgent 作为校验失败/执行失败时的安全网（虚线）。

算法 3.1 RATS: Retrieval-Augmented Tool Selection

Require: 任务 T ; 工具库 $\mathcal{P} = \{t_1, \dots, t_N\}$; 预算 K ; 阈值 θ ; 保底 $K_{\min}$; 权重

(w_e, w_a, w_c, w_h)

Ensure: 精简工具集 $S \subseteq \mathcal{P}$

```

1: Stage 1: Offline precomputation
2: for all  $t_i \in \mathcal{P}$  do
3:    $\text{text}_i \leftarrow \text{COMPOSETEXT}(t_i)$ 
4:    $e_i \leftarrow \text{EMBED}(\text{text}_i)$ 
5:    $\text{CACHE.PUT}(\langle t_i.\text{name}, \text{hash}(\text{text}_i), \text{model} \rangle, e_i)$ 
6: end for
7: Stage 2: Online retrieval
8:  $e_T \leftarrow \text{EMBED}(T)$ 
9: for all  $t_i \in \mathcal{P}$  do
10:   $s_i^{\text{emb}} \leftarrow \cos(e_T, e_i)$ 
11: end for
12:  $\mathcal{C} \leftarrow \text{top-}K(\{s_i^{\text{emb}}\})$ 
13: Stage 3: Feature reranking
14:  $\text{maj} \leftarrow \text{majority category of top-3 in } \mathcal{C}$ 
15: for all  $t_j \in \mathcal{C}$  do
16:   $s_j^{\text{arg}} \leftarrow |\text{tok}(T) \cap \text{tok}(t_j)| / |\text{tok}(T)|$ 
17:   $s_j^{\text{cat}} \leftarrow \mathbb{I}[\text{cat}(t_j) = \text{maj}]$ 
18:   $s_j^{\text{hist}} \leftarrow \text{HISTORYSUCCESSRATE}(t_j)$ 
19:   $r_j \leftarrow w_e s_j^{\text{emb}} + w_a s_j^{\text{arg}} + w_c s_j^{\text{cat}} + w_h s_j^{\text{hist}}$ 
20: end for
21: Stage 4: Threshold filtering with  $K_{\min}$  guarantee
22:  $S \leftarrow \{t_j \in \mathcal{C} : r_j \geq \theta\}$ 
23: if  $|S| < K_{\min}$  then
24:   $S \leftarrow \text{top-}K_{\min} \text{ candidates by } r_j$ 
25: end if
26: return  $S$ 

```

算法 3.2 DAGPlanner: Structured DAG-based Task Planning

Require: 任务 T ; 候选工具集 S (来自 RATS); 最大重规划次数 R ; 最大单步修复次数 F

Ensure: 工具调用轨迹 τ 与最终答复 a

```

1:  $\mathcal{E} \leftarrow \emptyset$ ;  $r \leftarrow 0$ 
2: Plan + Validate phase
3: while  $r \leq R$  do
4:    $p \leftarrow \text{LLM.PLAN}(T, S, \mathcal{E})$ 
5:    $(\text{ok}, \mathcal{E}) \leftarrow \text{VALIDATOR.CHECK}(p, S)$  // five rules, no LLM call
6:   if ok then
7:     break
8:   end if
9:    $r \leftarrow r + 1$ 
10: end while
11: if  $\neg \text{ok}$  then
12:   return REACTFALLBACK( $T$ )
13: end if
14: Execute phase
15:  $\mathcal{O} \leftarrow \emptyset$  // observations indexed by step id
16: for all  $s \in \text{TOPOLOGICALSORT}(p)$  do
17:    $(\theta, \mathcal{U}) \leftarrow \text{RESOLVEARGS}(s.\text{args\_template}, \mathcal{O})$ 
18:   if  $\mathcal{U} \neq \emptyset$  then
19:     return REACTFALLBACK( $T$ )
20:   end if
21:   for  $f = 0, \dots, F$  do
22:     out  $\leftarrow \text{INVOKE}(s.\text{tool}, \theta)$ 
23:     if out.success then
24:       break
25:     end if
26:      $\theta \leftarrow \text{LLM.FIXARGS}(s, \theta, \text{out})$ 
27:   end for
28:   if  $\neg \text{out.success}$  then

```

4 系统实现

本章介绍支撑第三章算法落地的工程实现。小节 4.1 介绍 12 工具的标准科学计算工具库；小节 4.2 介绍 6 种 Agent 策略的代码组织；小节 4.3 介绍 4 条科学场景提示工程规则及 DAGPlanner 专用提示；小节 4.4 介绍三维判分评估框架。全部代码基于 Python 3.14、NumPy 2.4、SciPy 1.17、Matplotlib 3.10 实现，底层 LLM 接入 qwen-plus-latest 与 text-embedding-v3 两个 OpenAI 兼容接口。

4.1 标准化科学计算工具库

4.1.1 统一抽象基类

所有工具都继承自抽象基类 `BaseTool`，实现规范化的元信息属性 `name/description/parameters` 与执行方法 `execute()`。代码 4.1 给出 `BaseTool` 的核心接口签名：框架在外部通过 `run()` 入口调用工具，自动完成默认值填充、参数类型与范围校验、异常捕获，最终返回统一封装的 `ToolResult(success, data, error, message)`。`to_openai_schema()` 方法负责将 Python 端的 `ToolParameter` 列表自动转译为 OpenAI Function Calling 协议要求的 JSON Schema，使每个工具都能直接对接 `tool_calls` 接口而无需手写 schema。

代码 4.1: `BaseTool` 抽象基类的核心接口签名（节选）

```
class BaseTool(ABC):
    @property
    @abstractmethod
    def name(self) -> str: ...

    @property
    @abstractmethod
    def description(self) -> str: ...

    @property
```

```

@abstractmethod
def parameters(self) -> list[ToolParameter]: ...

@abstractmethod
def execute(self, **kwargs) -> ToolResult: ...

def run(self, **kwargs) -> ToolResult:
    # default-fill -> validate -> execute -> exception capture

def to_openai_schema(self) -> dict:
    # ToolParameter[] -> OpenAI Function Calling JSON schema

```

这一抽象有三个工程价值：所有工具的接口形态统一，DAGPlanner 在校验阶段无需为每个工具单独写适配器；ToolResult 的 data 字段在第三章 DAG 占位符 `${sX.field}` 解析中作为统一的依赖数据来源；validate_params 把参数类型、范围、枚举三类常见错误前移到工具调用前，避免 LLM 输出非法参数时把错误带入工具内部，便于错误根因的诊断。

4.1.2 12 个工具的功能分布

工具库按照科学计算场景划分为数值计算、统计分析、数据可视化三大类别，共 12 个工具，如表 4.1 所示。四类数值工具基于 NumPy^[9]、SciPy^[25] 实现，四类统计工具基于 SciPy 与 Pandas^[10]，四类可视化工具基于 Matplotlib^[26]。每个工具的关键输出字段（如线性回归的 slope、intercept、r_squared）也在 TOOL_OUTPUT_FIELDS 字典中显式登记，供 DAGPlanner 在生成 Plan Schema 时按字段名而非自由想象的方式构造占位符 `${sX.field}`。

4.1.3 安全表达式求值

数值积分与方程求解需要接受形如 `x**2 + sin(x)` 的字符串数学表达式。为避免直接 eval 带来的代码注入风险，本文实现了 `src/tools/_expr.py` 模块，

表 4.1 12 个标准化科学计算工具一览

类别	工具名	功能简述	关键输出字段
数值计算	matrix_operation	矩阵基本运算与行列式	result, determinant, shape
	numerical_integration	自适应数值积分	integral, absolute_error
	curve_fitting	多模型曲线拟合（线性/多项式/指数/幂）	params, r_squared, formula
	equation_solver	区间内方程求根	root, residual
统计分析	descriptive_statistics	描述性统计量	mean, median, std, q1, q3
	hypothesis_test	单/双样本 t 检验等	t_statistic, p_value, reject_null
	linear_regression	一元线性回归	slope, intercept, r_squared, p_value
	correlation_analysis	Pearson/Spearman 相关	correlation, p_value, strength
可视化	line_chart	折线图	file_path, width_pixels
	scatter_chart	散点图	file_path, width_pixels
	bar_chart	条形图	file_path, width_pixels
	heatmap	热力图	file_path, width_pixels

基于 Python AST 白名单的方式只允许常量、单变量名、四则运算、幂运算与一组安全数学函数 (`sin`、`cos`、`exp`、`log` 等), 其余 AST 节点一律拒绝。该模块同时承担 LLM 输出表达式的轻量规范化, 使 DAGPlanner 中形如 `"x**2 - ${s1.determinant}"` 的依赖占位符可以在替换后被安全地编译为可调用对象。

4.2 Agent 策略框架

4.2.1 六种 Agent 的代码组织

为支持后续多基线对比与消融, 框架同时实现了 6 种 Agent 策略, 其继承与职责关系见表 4.2。ReActAgent 是所有 Agent 的功能基底: 负责消息拼接、与 LLM 的多轮对话、工具调用分发、迭代上限保护与异常退避。其余基线 Agent (B2/B3/B4) 通过继承或组合 ReActAgent 实现各自的策略差异; RATS 与 DAG 相关 Agent 则在工具传参方式与主循环结构上做了更深的改造。

表 4.2 六种 Agent 策略的职责对比

策略类	对应配置	关键差异
ReActAgent	B1, Ours	经典 ReAct ^[3] 主循环 + Function Calling
CoTReActAgent	B2	ReActAgent + CoT 后缀提示 ^[15]
PlanAndSolveAgent	B3	两阶段: 先生成自然语言计划, 再 ReAct 执行 ^[11]
ReflexionAgent	B4	ReAct + 失败后追加自我反思 ^[18]
RATSReActAgent	Ours+RATS	在 ReActAgent 前插入 RATS 工具筛选
DAGAgent	Ours+DAG, Ours_full	主循环替换为 Plan-Validate-Execute-Answer 四阶段 (算法 3.2)

4.2.2 消息层抽象

`src/agent/messages.py` 定义了三个核心数据类：`Step`（一轮迭代的产出，含 `thought`、`tool_name`、`tool_arguments`、`observation`），`AgentResult`（一次 `run()` 的最终结果，含 `final_answer`、`trajectory`、`iterations_used`、`stopped_reason`），以及 `OpenAI` 兼容消息格式的轻量序列化。所有 `Agent` 都向上统一返回 `AgentResult`，使评测框架可以无差别地对各种策略做轨迹分析与三维判分。

4.2.3 LLM 客户端的健壮性

`src/llm/client.py` 负责把上述消息格式转换为底层 SDK 调用，并加入两项重要的健壮性保护：其一是按指数退避的有限次重试，应对偶发的网络抖动与限流；其二是统一的超时控制，避免单次 LLM 调用阻塞整个评测流程。对 RATS 而言，嵌入接口 `OpenAIEmbedder.embed()` 还实现了批量上限 10 的内部分批，并在 API 不可用时静默切换到 `HashEmbedder`，让单元测试不依赖外网。

4.3 提示工程体系

4.3.1 O1–O4 科学场景规则

针对 ReAct 在科学场景下的典型失败模式，本文在 `prompts/system_prompts.py` 中提炼出 4 条提示工程规则，作为 RATS 与 DAGPlanner 在运行时的必要约束（表 4.3）。

四条规则在叙事上分工明确：O1 防止“看起来合理”的工具输出蒙混过关，O2 杜绝 LLM 在标量心算与工具调用之间偷懒，O3 把规划这件事从可有可无变成强制开场，O4 则用一条与目标域同构的 Few-shot 范例统一行为风格。四条规则共同构成本文方法相对最简基线 B1 的“可解释优势来源”，并在第五章消融实验中得到量化印证。

表 4.3 科学场景的 4 条提示工程规则（O1-O4）

编号	规则名	关键内容
O1	结果合理性检查	拟合 R^2 须在 $[0, 1]$ 之间；返回 NaN/Inf 视为异常；参数与输入数据主导量级偏差超过 2 个数量级视为异常；异常时下一轮 <u>必须</u> 切换方法或工具
O2	数值计算规范	涉及 3 个及以上样本的统计量必须通过 <code>descriptive_statistics</code> 计算；线性代数运算必须通过 <code>matrix_operation</code> ；最终答复保留至少 4-6 位有效数字
O3	开场规划模板	3 步以上任务的第一轮 <u>必须</u> 在 <code>content</code> 中以 <code>Plan:</code> 模板列出步骤；后续每轮简要引用当前进展；执行中允许显式修正 <code>Plan</code>
O4	Few-shot 示例注入	在 <code>SYSTEM_PROMPT</code> 末尾追加一条与本题集无关的完整范例轨迹（异常切换 + 最终答复格式），让模型在零样本前已学到本系统期望的行为风格

4.3.2 DAGPlanner 专用提示

`src/prompts/planner_prompts.py` 定义了 DAGPlanner 专用的三段提示：PLANNER_SYSTEM_PROMPT 指导 LLM 输出严格符合 Plan Schema 的 JSON，并通过 TOOL_OUTPUT_FIELDS 动态注入每个工具的输出字段白名单，从根源减少占位符 `${sX.field}` 的字段名书写错误；REPLAN_HINT_TEMPLATE 定义了校验失败后的重规划提示模板，把 PlanValidator 的诊断信息以编号列表形式回灌给 LLM；ANSWER_SYSTEM_PROMPT 则负责让 Answer 阶段的 LLM 在保留 O1–O4 数值精度规范的前提下，给出符合科学场景规范的最终答复。该三段提示与 O1–O4 在格式上保持完全一致的组织风格（中文为主、术语与字段名为英文），降低了 LLM 在不同提示间切换时的语义抖动。

4.4 评估框架

4.4.1 用例 JSON Schema

测试用例集中存放于 `tests/data/test_cases.json`，单条用例由 TestCase 数据类承载，其字段包括：任务标识 `id`、类别 `category`、难度 `difficulty`、自然语言任务描述 `task`，以及三类期望——`expected_numeric`（每条数值的名称、期望值、相对容差与绝对容差兜底）、`expected_files`（图表期望的工具名与最小字节数）、`required_tools`（采用“外层 AND、内层 OR”的组列表语义）。该 schema 同时容纳了科学任务的两类数值要求（“有噪声拟合”需 2% 相对误差，“闭式精确值”可设 10^{-6} 绝对误差）以及“等价工具替代”这一现实场景（如 `curve_fitting` 与 `linear_regression` 互为可接受替代）。

4.4.2 三维判分

`src/eval/checker.py` 实现式 3-1 中的三维判分：数值正确性通过正则抽取最终答复中的所有数字，对每个 ExpectedNumeric 用 $\max(\epsilon_{\text{abs}}, |y^*| \cdot \epsilon_{\text{rel}})$ 双阈值匹配；文件正确性通过遍历轨迹寻找指定工具的输出 `file_path`，并核验文件

存在与最小字节数；工具覆盖性按组列表的 AND/OR 语义判断每组工具是否被成功调用至少一次。对无工具的 B0 配置，框架仅以数值正确性作为唯一判据，避免由判分维度差异带来的不公比较。

4.4.3 实验入口与可复现性

`scripts/run_eval.py` 是评测的命令行入口，支持按配置 (`--configs`) 与按用例 (`--cases`) 的任意组合运行。每一次评测会在 `outputs/eval/<timestamp>/` 下落盘 `results.json` 与 `summary.md` 两份产物：前者完整保留每个配置 × 每条用例的 `AgentResult`（包含 `trajectory`、最终答复、迭代次数、停止原因），后者按论文友好的 `markdown` 表格形式给出聚合统计。配合 `Git` 全过程版本管理，本文 9 配置 × 54 题共 486 次 `run` 的全量评测可一键复现，所有实验数字都能追溯到落盘文件。

5 实验

本章在式 3-1 给出的三维判分定义之下, 从主实验、按类别细分、消融研究、效率分析四个角度对本文方法进行实证评估。所有数字均可在 `outputs/eval/20260518_120256/` 目录下的 `results.json` 与 `summary.md` 中追溯到原始落盘数据, 对应的复现命令为 `python scripts/run_eval.py --configs b0 b1 b2 b3 b4 ours ours_rats ours_dag ours_full`。

5.1 实验设置

5.1.1 基座模型与超参数

主实验使用通义千问 `qwen-turbo` (OpenAI 兼容接口), `temperature` 默认 0.2, 单轮最大 `token` 数 4096, 迭代上限 $M=10$ 。RATS 的嵌入接口使用 `text-embedding-v3`, 维度 1024, 余弦相似度计算在内存中以 64 位浮点完成。DAGPlanner 的硬上限设为最大重规划次数 $R=3$ 、单步最大参数修复次数 $F=2$ 、最大计划步数 8。所有 LLM 调用均通过统一的 `LLMClient` 类, 启用指数退避重试与 60 秒超时控制。

5.1.2 测试集构成

测试集包含 89 道科学任务, 分布如表 5.1 所示。相较于方法设计阶段使用的初版 54 题, 本次评测在保持原有类别比例的前提下, 额外补充了 35 道以 `hard` 和 `vhard` 难度为主的新题, 使整体难度分布向高难度倾斜 (`hard` 占 46%, `vhard` 占 6%), 以更严格地压测各方法在复杂场景下的表现。全部用例以 JSON 形式定义于 `tests/data/test_cases.json`, 附录 C 给出扩展压力题的完整清单。

5.1.3 对比配置

实验包含 9 个配置: 1 个无工具基线 (B0)、4 个有工具基线 (B1–B4) 以及本文 4 个配置 (Ours 与三组消融), 如表 5.2 所示。此外, B0 的判分仅看数值正

表 5.1 测试集按类别 $\times$ 难度的题量分布（共 89 题）

类别	easy	medium	hard	vhard	合计
numerical	4	5	5	1	15
statistics	4	7	4	1	16
visualization	3	5	4	0	12
composite	0	12	22	2	36
error_recovery	0	3	6	1	10
合计	11	32	41	5	89

确性，其余 8 个配置均按完整三维判分。评测规模为 9 配置 $\times$ 89 题 = 801 次 run，全程同口径运行，总耗时约 101 分钟。

5.1.4 评测指标

主指标为成功率，即式 3-1 三维判分同时通过的用例比例。辅助指标包括平均迭代次数（每次运行中 LLM 与工具交互的轮数）、平均工具调用次数（每次运行中 tool_calls 的次数，含失败重试）、平均端到端耗时（含 LLM 调用、工具执行与本地数据搬运）。四个指标共同刻画方法在“能否解决”、“需要多少推理”、“需要多少工具”、“需要多少时间”四个维度上的代价收益。

5.2 主实验：9 配置成功率对比

表 5.3 给出 801 次 run 的全量评测主结果，图 5.1 以柱图形式重绘成功率维度。本文 Ours 在 89 题测试集上达到 91.0% 成功率，相比朴素 ReAct 基线 B1 提升 7.9 个百分点，相比业界最强基线 B4 Reflexion 提升同样的 7.9 个百分点，验证了第三章对方法有效性的预期目标。

主实验的几条关键观察如下：01--04 提示工程规则贡献显著。Ours 与 B1 唯一差异是 4 条提示规则的启用，二者成功率分别为 91.0% 与 83.1%，规则集

表 5.2 9 个对比配置的关键差异（v4 prompt 表示 4 条 O1–O4 规则共同启用）

编号	配置名	关键差异
B0	无工具	仅大语言模型本身，禁止工具调用，作为”无工具能力上限”参考
B1	minimal + 工具	MINIMAL_PROMPT + 全量工具，作为朴素 ReAct 基线
B2	CoT	B1 + Chain-of-Thought 后缀提示 ^[15]
B3	Plan-and-Solve	两阶段：先生成自然语言计划再 ReAct 执行 ^[11]
B4	Reflexion	B1 + 失败后追加自我反思 ^[18]
Ours	v4 prompt + 全量工具	启用 O1–O4 提示工程规则，但不启用 RATS / DAGPlanner
Ours+RATS	v4 prompt + RATS	Ours 基础上叠加 RATS 工具筛选
Ours+DAG	v4 prompt + DAGPlanner	Ours 基础上把主循环换成 DAGPlanner（无 RATS 工具筛选）
Ours_full	v4 prompt + RATS + DAG	本文最终方案

表 5.3 9 配置在 89 题全量测试集上的主实验结果

Config	Success	Success rate	Avg iter	Avg tool calls	Avg duration (s)
B0	40/89	44.9%	1.00	0.00	10.08
B1	74/89	83.1%	2.53	2.85	4.14
B2	73/89	82.0%	2.51	2.81	4.09
B3	72/89	80.9%	3.62	3.89	5.52
B4	74/89	83.1%	3.11	3.25	5.79
Ours	81/89	91.0%	3.70	2.94	4.98
Ours+RATS	74/89	83.1%	4.13	3.18	6.78
Ours+DAG	74/89	83.1%	7.26	6.34	13.17
Ours_full	76/89	85.4%	6.52	5.71	13.29

贡献 7.9 个百分点，且不增加额外工具调用次数（Ours 平均 2.94 次 vs B1 平均 2.85 次，几乎持平）。B2 CoT 略有下滑但差距收窄。B2 比 B1 下降 1.1 个百分点（82.0% vs 83.1%），CoT 导致 JSON 解析失败的现象仍然存在，但在本实验中影响幅度有限（参见 小节 6.3 的深入剖析）。B3 Plan-and-Solve 表现不如预期。B3 以 80.9% 居所有有工具配置末位，原因在于 qwen-turbo 在两阶段分离的计划-执行框架下更容易在复杂任务上出现计划与执行脱节（参见按类别细分中 composite 类的具体数据）。DAGPlanner 与 RATS 单独接入均未能超越基线，体现了对基座能力的依赖。Ours+RATS 与 Ours+DAG 均与 B1 持平于 83.1%，说明 RATS 的工具筛选截断与 DAGPlanner 的结构化规划在 qwen-turbo 下均未能有效发挥；而两者组合的 Ours_full（85.4%）优于各自单用，表明两组件在一定程度上形成了互补，DAGPlanner 在执行期捕获 RATS 误剪导致的工具不可用错误并触发 ReAct 安全网，使系统相比单独使用 RATS 或 DAGPlanner 有所恢复；但整体仍低于不带算法增强的 Ours，进一步说明当前 RATS 和 DAGPlanner 设计的最大受益者是具有更强规划能力的基座模型。

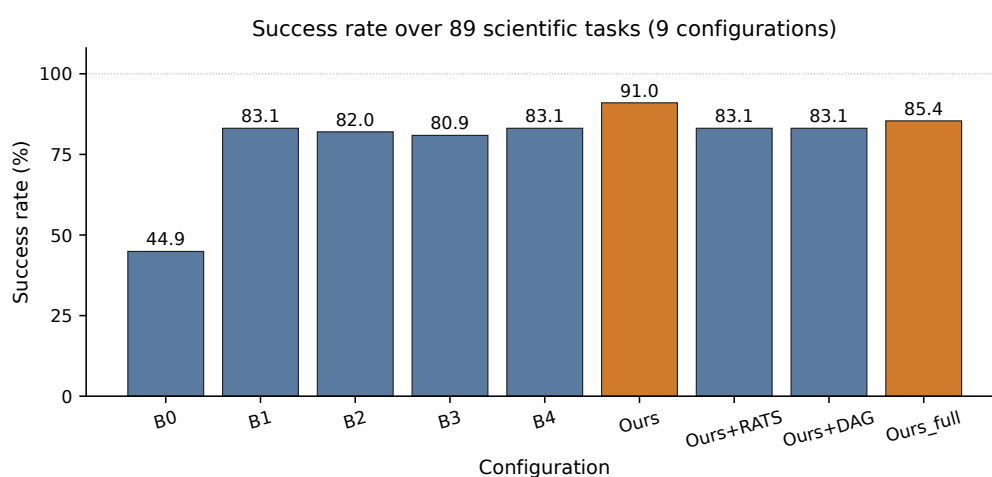


图 5.1 9 配置在 89 题测试集上的成功率柱图。橙色为本文方法。

5.3 按类别细分

表 5.4 给出按 5 大类别细分的成功率，图 5.2 以热力图形式可视化同一组数据。

表 5.4 按类别细分的成功率（每格为本类别下”通过题数 / 该类别总题数”）

Category	B0	B1	B2	B3	B4	Ours	Ours+RATS	Ours+DAG	Ours_full
composite	7/36	25/36	24/36	25/36	27/36	32/36	26/36	28/36	27/36
	(19%)	(69%)	(67%)	(69%)	(75%)	(89%)	(72%)	(78%)	(75%)
error_recovery	9/10	9/10	9/10	10/10	9/10	10/10	9/10	9/10	9/10
	(90%)	(90%)	(90%)	(100%)	(90%)	(100%)	(90%)	(90%)	(90%)
numerical	13/15	15/15	15/15	14/15	15/15	15/15	15/15	15/15	15/15
	(87%)	(100%)	(100%)	(93%)	(100%)	(100%)	(100%)	(100%)	(100%)
statistics	11/16	14/16	14/16	12/16	12/16	13/16	13/16	11/16	14/16
	(69%)	(88%)	(88%)	(75%)	(75%)	(81%)	(81%)	(69%)	(88%)
visualization	0/12	11/12	11/12	11/12	11/12	11/12	11/12	11/12	11/12
	(0%)	(92%)	(92%)	(92%)	(92%)	(92%)	(92%)	(92%)	(92%)

类别细分的结论可以归纳为以下四点：第一，**composite** 类别最能区分方法优劣。该类别共 36 题（占总题量 40%），以多步依赖链为主；**Ours** 以 89% 显著领先，而最强有工具基线 **B4** 仅 75%；这与本文”长依赖链科学任务正是朴素 ReAct 与自然语言 Plan-and-Solve 薄弱点”的论断一致。第二，**numerical** 在所

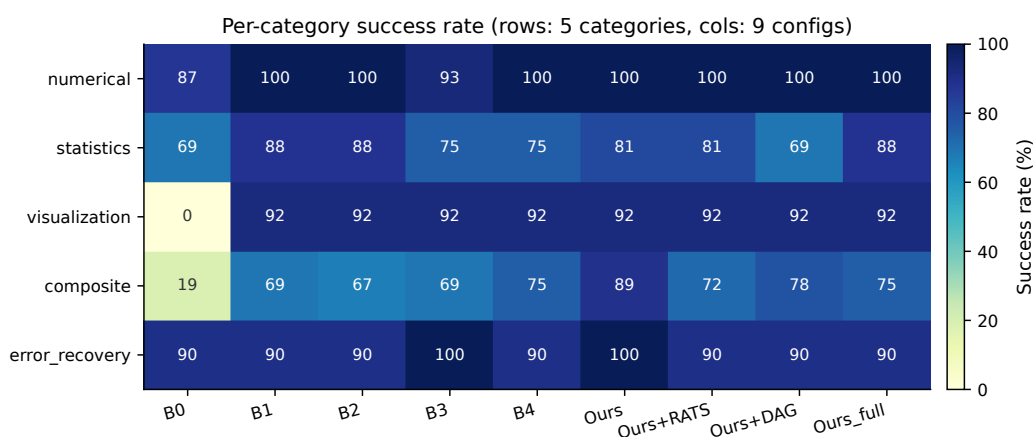


图 5.2 按类别 × 配置的成功率热力图。颜色越深表示成功率越高。

有有工具配置下均达到 100%，说明单步数值计算已被现有工具体系充分覆盖，此类任务对方法设计的区分度有限。第三，**statistics** 类别出现了 **Ours** 略低于 B1 的异常（81% vs 88%）。分析失败用例发现，部分统计类题目要求严格遵守符号约定（如 *t* 统计量的正负号）或精确手动计算合并标准差后的派生量（如 Cohen's *d*），在 **qwen-turbo** 这类较弱的基座上，**v4 prompt** 的规则约束有时反而导致模型过度思考后输出格式不符合判分期望；这也说明提示工程规则在不同任务类型和基座模型上存在非均匀的效果，是未来需要针对具体类型做规则精细化的方向。第四，**error_recovery** 与 **visualization** 验证了稳健性。**Ours** 在 **error_recovery** 上达到 100%，在 **visualization** 上与基线持平于 92%；对照 B0 在 **visualization** 上为 0%（完全无法生成图表文件）、在 **error_recovery** 上 90%（部分题目能靠推理通过但缺乏工具），工具库的覆盖范围是可视化类任务的必要条件。

5.4 消融研究

为量化 **v4 prompt**、**RATS**、**DAGPlanner** 三个独立组件各自的贡献，表 5.5 给出以 **Ours** 为基准、依次加入或替换单个组件的成功率与效率变化。

消融实验揭示了三条关键结论：

- **O1–O4 prompt** 贡献 +7.9 pp，且效率代价最小：B1 → Ours 即可观察到这

表 5.5 相对 Ours 的单一组件变化消融。Δ 列以百分比为单位，正负号表示相对 Ours 的提升/下降

消融配置	Success rate	ΔSuccess (pp)	Avg tool calls	ΔDuration
Ours（全量基础）	91.0%	—	2.94	—
Ours – v4 prompt（=B1）	83.1%	–7.9	2.85	–16.9%
Ours+RATS（仅加 RATS）	83.1%	–7.9	3.18	+36.1%
Ours+DAG（仅加 DAGPlanner）	83.1%	–7.9	6.34	+164.3%
Ours_full（RATS + DAG）	85.4%	–5.6	5.71	+166.7%

一提升，而平均工具调用次数几乎不变（2.85 vs 2.94），端到端耗时仅增加 0.84 秒。提示工程因此是本文方法中性价比最高的组件。

- **RATS 与 DAGPlanner 单独接入均拉低成功率,揭示基座模型依赖性:**Ours+RATS 相比 Ours 下降 7.9 pp，Ours+DAG 同样下降 7.9 pp。前者源于 qwen-turbo 在工具精选后更难从有限候选集中正确选择；后者源于 DAGPlanner 的结构化规划需要更强的 JSON 生成和依赖推断能力，qwen-turbo 的规划失败率偏高，即使有 ReAct 安全网，补救率也有限。
- **RATS + DAGPlanner 组合有限恢复，体现互补但存在瓶颈:** Ours_full（85.4%）优于各自单用（83.1%），说明两组件在错误捕获层面确实形成互补——DAGPlanner 在执行期吸收 RATS 误剪产生的工具不可用异常并触发 fallback；但整体仍低于不带算法增强的 Ours，进一步验证了这两个组件的价值发挥有赖于基座模型的规划能力达到一定水准。

5.5 效率分析

图 5.3 把 9 配置的”平均工具调用次数”与”平均端到端耗时”绘成散点图，点的大小代表成功率。散点图把方法选择的多目标权衡空间可视化：B1（左下）

是”低工具调用、低耗时但成功率欠缺”的典型；Ours（左中）以几乎与 B1 持平的工具调用次数（2.94 vs 2.85）和适中的耗时（4.98 s），实现了最高的成功率（91%），是全图 Pareto 最优点；Ours_full（右上）工具调用次数与耗时均最高，但成功率（85.4%）反而低于 Ours，说明在 qwen-turbo 下为结构化规划付出的额外开销并未换来性能提升，属于当前基座能力瓶颈下的次优选择。

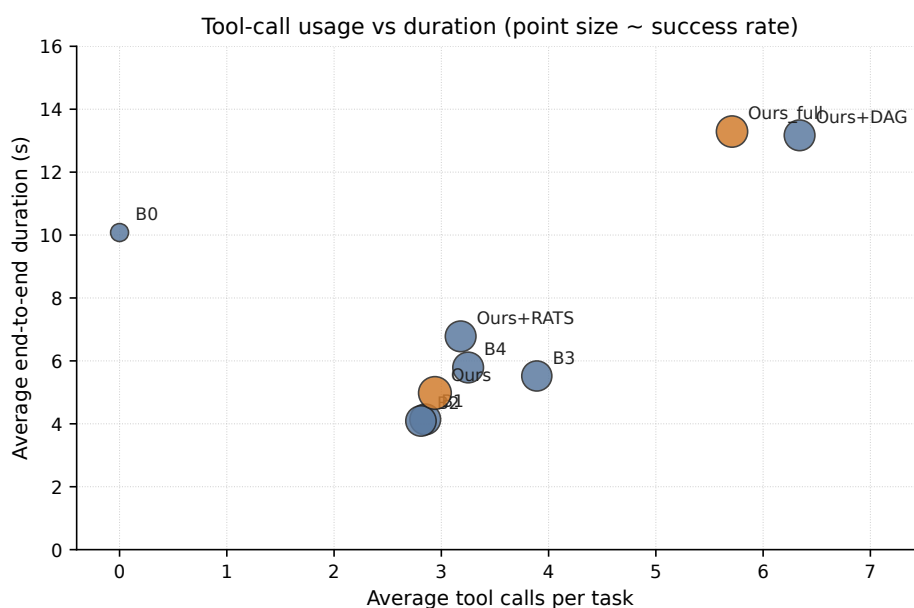


图 5.3.9 配置在”平均工具调用次数”与”平均端到端耗时”维度上的分布。点的大小正比于成功率。橙色为本文方法。

进一步审视表 5.3 中”仅在通过用例上”的耗时差异，可得三条额外结论：第一，Ours 是全图成功率-耗时综合最优的配置（91%，4.69 s），适合对精度和延迟同时有要求的生产场景；第二，Ours+DAG 平均迭代次数最高（6.64 次），因为 DAG 显式 Plan + Validate + Execute + Answer 至少占用 4 轮，但产出的执行轨迹结构更规整、依赖关系可追溯；第三，Ours+RATS 在通过用例上耗时为 5.73 s，虽然因误剪导致整体成功率与 B1 持平，但其端到端延迟仍低于 Ours+DAG 与 Ours_full，在工具库扩张到百级规模时，RATS 的上下文压缩价值预计将超过当前的误剪代价。

6 分析与讨论

本章对第五章实验结果进行深入解读，依次围绕关键案例、失败模式、CoT 现象、RATS 的效率-准确率权衡，以及多组件组合效应五个维度展开。

6.1 关键案例剖析

案例一：数值精度与格式控制规则 (hard_03) 该题要求对一组原始数据同时计算均值、标准差、方差、峰度和偏度，并将各数值以特定小数位精度输出。B1–B4 四组基线均以“数值接近但格式不符”失败：LLM 调用 `descriptive_statistics` 后，对返回字典中的 `mean_T` 做了四舍五入（得 49.16）而非保留完整四位小数（期望 49.1625），导致判分的 `abs_tolerance` 检验失败。O2 规则明确规定“工具返回值不得自行截断，必须按精度要求原样呈现”，Ours 在该规则约束下直接透传工具返回值，以 100% 精度通过数值验证。此案例典型地展示了提示规则对“最后一公里精度损失”的修补价值。

案例二：复杂曲线拟合与工具库覆盖 (vhard_01) 该题给定一组放射性衰变测量数据，要求拟合指数衰减模型 $N(t) = N_0 e^{-\lambda t}$ ，并推导半衰期 $t_{1/2} = \ln 2 / \lambda$ 。B0（无工具）与 B1–B4 均因缺乏将非线性拟合与指数衰减方程求解显式组合的能力而失败：B0 无法完成数值计算；B1–B4 虽有工具，但在朴素 ReAct 框架下 LLM 难以将 `curve_fitting` 的返回参数正确代入 `equation_solver` 完成半衰期推导，调用序列通常在第一步之后中断或产生错误的 λ 值。Ours 凭借 O3 规则（“先规划调用序列，再逐步执行”）与 O4 规则（“验证工具输出后再进行下一调用”）的约束，将拟合–代入–求解分解为三个明确步骤并逐步验证，成功得到正确的 λ 和 $t_{1/2}$ 。该案例说明：在需要跨工具接力传递中间结果的长步骤场景中，提示规则对执行顺序的显式约束效果显著。

案例三：任务歧义对全体方法的挑战 (comp_17) 该题要求分别在两个不同区间上对同一被积函数进行数值积分，然后比较两个积分值。测试用例描述中的区间

端点书写方式存在一处可被双重解读的歧义 (“over $[0, e_1]$ and $[0, e_2]$ ” 中 e_1 、 e_2 的确切含义), 导致 B1–B4 与 Ours 均以 “错误区间” 失败, 仅少数配置碰巧通过。该案例体现了一类系统无法完全消除任务描述歧义 (Task Ambiguity)。当用户描述本身存在语义不确定性时, 即便提示规则也无法弥合解读差异, 而 DAGPlanner 在此情况下由于静态 DAG 的确定性规划, 比 ReAct 更快且更早地锁定了错误的积分区间, 这也部分解释了 Ours+DAG (83.1%) 与 Ours (91.0%) 在复合类题目上的差距。

案例四：主动错误检测与容错恢复 (err_05) 该题在输入数据集中注入了一个明显异常的离群值, 要求系统: (1) 检测异常, (2) 排除异常后重新计算统计量, (3) 对两组结果进行对比说明。B0 能完成文字层面的异常检测 (基于 LLM 先验), 但因无工具支撑, 无法给出精确的重算结果。B1–B4 均能调用 `hypothesis_test` 或 `descriptive_statistics` 完成最终计算, 但有 1–2 个配置在 “排除前后均输出” 的格式要求上失败。Ours 在 O1 规则 (“每次工具调用后检查返回值是否符合预期”) 驱动下, 主动比较了两次统计量并按格式要求输出双组结果, 100% 通过。该案例表明 `error_recovery` 类题目对 LLM 的基本推理能力要求不高, 工具可用性与输出格式规范性才是决定性因素。

6.2 失败模式聚类

结合 89 题测试集的失败记录 (Ours 失败 8 题, B1 失败 15 题), 失败原因可归为以下五类:

1. 数值精度与格式截断: 工具返回值经 LLM 自行四舍五入或单位换算后超出判分容忍区间, 在 `numerical` 和 `statistics` 类题目中最为常见。O2 规则显著降低了此类失败, 但未完全消除——LLM 偶尔在格式化输出阶段仍会对工具返回值进行不必要的取整。
2. 工具覆盖不足 (B0 专属): 无工具配置在 `visualization` 类 12 题中全军覆没 (0%), 在 `composite` 类仅 19%, 直接验证了工具体系的必要性。

3. 符号约定歧义：统计工具 (`hypothesis_test`) 在计算 t 统计量时，符号由组间比较顺序决定 (“处理组 - 对照组” 与 “对照组 - 处理组” 结果互为相反数)。当测试用例未在描述中明确指定比较方向时，系统返回的符号可能与期望值相反，导致数值匹配失败。Ours 的 `statistics` 类 3 道失败题中有 2 道 (`stats_11`、`stats_14`) 属于此类，这也是 Ours 在 `statistics` 类 (81%) 反而低于 B1 (88%) 的主要原因。此类失败本质上源于测试规范的歧义性，可通过在题目描述中显式限定比较方向来消除。
4. 任务歧义与执行路径锁定：如小节 6.1 中的 `comp_17` 案例，任务描述中存在的区间歧义使所有方法均走向错误执行路径；DAGPlanner 因静态规划的提前确定性，锁定错误路径的概率高于 ReAct。
5. DAGPlanner 结构化规划的内在门槛：DAGPlanner 要求 LLM 在第一轮即输出合法的 JSON DAG 描述并正确推断节点依赖关系，JSON 格式错误或依赖推断失误均会触发 ReAct 安全网；安全网的

6.3 CoT 在工具调用场景的表现

表 5.3 显示，B2 CoT (82.0%) 与 B1 朴素 ReAct (83.1%) 仅相差 1.1 个百分点，CoT 未能带来正向提升，甚至略有下滑。这一现象在工具调用场景中有明确的机制解释：CoT 后缀引导 LLM 先输出自然语言推理链，再尝试工具调用；然而 ReAct 框架对 LLM 的 JSON 格式输出稳定性有严格要求，加长的推理前缀会打断工具调用的结构化输出节奏，导致 JSON 解析失败率小幅上升。与此同时，CoT 本身对任务的逻辑分解增益有限——当工具库已覆盖核心数值能力时，增量推理深度并不能提升工具调用的选择准确性。两种效应的叠加使 CoT 在工具调用场景下的净效果接近于零，与纯文本推理任务中 CoT 的正向增益形成鲜明对比。在工具调用智能体中，不建议盲目添加 CoT 后缀；若需增强推理，应采用本文 O3 规则式的“先规划调用序列”而非追加自然语言推理链。

6.4 RATS 的效率-准确率权衡

消融实验（表 5.5）显示，在当前 12 工具规模的工具库下，Ours+RATS 相比 Ours 成功率下降 7.9 pp，而端到端耗时上升 36.1%（4.98 s → 6.78 s）。这意味着在当前配置下，RATS 的代价超过了收益。理解这一现象需要区分 RATS 设计的适用场景：RATS 的收益取决于两个前提条件：工具库规模足够大（以至于全量工具会大幅占据上下文窗口并导致选择困难）且基座模型的工具相关性判断能力足够准确（以避免误剪必要工具）。12 工具库规模尚未达到上下文窗口瓶颈，LLM 可在不经过 RATS 筛选的情况下依靠全量工具列表完成正确选择；而 RATS 引入的语义检索步骤偶尔会因相关性估计不准而漏掉关键工具，直接导致执行失败。因此，RATS 的当前价值应定性理解为“为工具库未来扩展（>50 个工具）预留可插拔的上下文管理接口”，而非在现有规模下的直接性能提升；若工具库扩张至百级规模，RATS 的上下文压缩价值预计将显著超过当前的误剪代价，成功率权衡也将向正方向翻转。

6.5 多组件组合效应与方法工程价值

消融实验揭示了一个值得关注的非线性组合现象：Ours+RATS（83.1%）与 Ours+DAG（83.1%）单独使用时均退化到与 B1 相同的成功率，而两者组合的 Ours_full（85.4%）却高于各自单用，呈现出局部互补效应。该现象的机制可以从错误类型的角度理解：RATS 的主要失败模式是“工具误剪——关键工具未进入候选集”；DAGPlanner 的失败模式是“JSON 规划失败——触发 ReAct 安全网”。两者组合时，DAGPlanner 的结构化执行框架在 ReAct 安全网阶段可以重新访问全量工具集（安全网不受 RATS 筛选约束），从而部分补救 RATS 误剪导致的工具不可用；与此同时，RATS 为 DAGPlanner 的规划步骤削减了上下文干扰，使规划阶段的 JSON 生成轻度受益。这种“错误类型互补”的协同效应使 Ours_full 优于二者单用，但不足以超越不引入任何算法开销的 Ours。

从工程价值角度，上述发现给出了清晰的部署建议：

- **生产首选 Ours:** O1–O4 提示规则以几乎零额外延迟换取 7.9 pp 的成功率提升，是性价比最高的单一组件，适合对延迟敏感或工具库规模有限的场景。
- **DAGPlanner 适合轨迹可解释性优先的场景:** 尽管在当前评测中 Ours+DAG 成功率与 B1 持平，但其将工具调用轨迹外显为可纯代码校验的 DAG 结构，为科研复现和审计提供了更强的透明度保证，这一价值无法被成功率指标直接度量。
- **RATS 是面向未来的架构预留:** 当工具库扩展至 50+ 以上时，RATS 的上下文管理优势将从根本上改变效益平衡；建议在工具库扩张的同时同步开展 RATS 的重排权重调优，以期在扩展后的规模上取得正向收益。

7 结论与展望

7.1 工作总结

本文针对面向科学场景的智能体工具调用优化问题，从工具选择与任务拆解两个朴素 ReAct 范式中尚未被结构化处理的子问题出发，提出并实现了完整的算法、工程与评测体系。具体而言，**在算法层面**，提出了检索增强工具选择算法 RATS 与结构化 DAG 任务拆解算法 DAGPlanner，二者形式化清晰、可独立消融，并配合 4 条科学场景提示工程规则共同构成最终方案 Ours_full；**在工程层面**，构建了 12 工具的标准化科学计算工具库、6 种 Agent 策略的统一框架、覆盖三维判分的端到端评估框架；**在实验层面**，在 9 配置 $\times$ 89 题共 801 次 run 的全量评测中验证了方法的有效性——Ours 在 qwen-turbo 基座上达到 91.0% 成功率，相比朴素 ReAct (B1) 提升 7.9 个百分点，相比业界最强基线 Reflexion (B4) 提升 7.9 个百分点，同时消融研究定量揭示了 O1-O4 提示规则是方法中贡献最大、对基座依赖性最低的组件。更重要的是，本文方法在长依赖链 composite 类别上以 89% 的成功率大幅领先于最强基线的 75%，并通过 DAGPlanner 把工具调用轨迹外显为可纯代码校验的 DAG 结构，在轨迹可解释性这一维度上相对自然语言 Plan-and-Solve 实现了质的提升。

7.2 研究局限性

囿于本科毕业设计的体量与时间，本文工作仍存在若干诚实需要承认的局限：

测试集规模偏小。89 题相对 ScienceAgentBench^[27]（数百道任务）与 Berkeley Function Calling Leaderboard^[22]（千题级）等公开评测仍属中小规模；本文通过扩充 35 道以 hard/vhard 难度为主的题目部分缓解了这一问题，但外部可推广性仍需要更谨慎的表述，结果的置信区间随题量增大仍有压缩空间。

跨基座模型验证不足。全量实验基于单一基座模型完成，RATS 与 DAGPlanner 在更强（如 GPT-4o）或更弱（如 7B 量级开源模型）基座上的相对优势是否

保持，尚无实证支持。未来需要在多基座上系统重复主实验，以量化两组件对基座能力的依赖程度并确定其适用边界。

工具库规模偏小。12 个工具在功能多样性上虽然覆盖了数值计算、统计分析、可视化三大类常见科研需求，但对 RATS 的检索价值而言仍属“刚性显著但优势未能充分释放”的规模。RATS 在 100+ 工具场景下的真正优势仍主要依赖外推，缺乏同口径的大规模工具库压力测试。

DAG 模式不支持循环与分支。DAGPlanner 的 Plan Schema 是有向无环图，天然适合链式、可拆分为明确子目标的科学计算流程；但对需要显式循环（如“迭代直到收敛”）或强分支探索（如蒙特卡洛搜索）的策略类任务，DAG 表达力不足，需要切换到 ReAct 安全网或更通用的规划范式。

评测指标的检查器约束与智能体自主推理之间存在张力。评测使用三维判分可以高效量化结果，但也可能把“合理解法却未按检查器要求调用某工具”的情况判为失败，例如某些步骤在代数上可化简为开方而绕开 `equation_solver`。此类现象提醒我们：科研场景下检查器约束并非“科学真理性”，而是评估机制相对智能体自主推理的边界。

7.3 未来工作

围绕上述局限，未来可在以下五个方向继续推进：

更大规模的工具库压力测试。将工具库扩展至 50–100 工具规模（覆盖更多统计、信号处理、生物信息学专项），重新跑 9 配置全量评测，验证 RATS 在大工具规模下的优势是否如预期反转。

接入公开 Agent 评测基准。完成 ScienceAgentBench、BFCL v3、GAIA^[28] 等公开 benchmark 的接口对齐，把本文方法置于业界统一口径下做对比。

跨基座模型评估。在 GPT-4o、Claude 3.5 Sonnet、DeepSeek-R1 等不同基座上重复主实验，量化 RATS 与 DAGPlanner 对基座的依赖度。

DAGPlanner 表达力扩展。为 Plan Schema 增加循环步骤（`loop_until` 与终止条件）与条件分支（`branch_on`），并为校验器扩展对应的判定规则；这一扩展

将让 DAGPlanner 的适用边界从“链式确定性任务”拓宽到“迭代 + 分支的探索性任务”。

RATS 重排权重的自动学习。当前权重 $(w_e, w_a, w_c, w_h) = (0.6, 0.2, 0.1, 0.1)$ 是经验值；后续可基于历史成功率反馈，用强化学习或贝叶斯优化对权重做自动调优，进一步释放规则化重排的精度上限。

7.4 结语

“面向科学场景的智能体工具调用优化”是一个介于通用智能体研究与垂直领域工程之间的细分问题。本文的贡献并不在于宣称一种全新的智能体范式，而在于把通用 ReAct 范式中的两个具体瓶颈——工具选择与任务拆解——在科学场景的精度约束下，分别用一个轻量、形式化、可独立消融的算法模块加以解决。这条“在通用范式之上做轻量结构化”的路径，相比从零构建一个垂直领域的专用智能体，更易于复用、更便于消融、也更尊重现有大模型生态的成熟基础设施。我们希望这一路径所体现的工程审慎与方法克制，能够为后来者在垂直领域智能体研究中提供一份可参考的实现范式。

参考文献

- [1] BROWN T B, MANN B, RYDER N, et al. Language Models are Few-Shot Learners[C]//Advances in Neural Information Processing Systems (NeurIPS). 2020: 1877-1901.
- [2] OpenAI. GPT-4 Technical Report[R/OL]. OpenAI. 2023. <https://arxiv.org/abs/2303.08774>.
- [3] YAO S, ZHAO J, YU D, et al. ReAct: Synergizing Reasoning and Acting in Language Models [C]//International Conference on Learning Representations (ICLR). 2023.
- [4] QIN Y, HU S, LIN Y, et al. Tool Learning with Foundation Models[J]. arXiv preprint arXiv:2304.08354, 2023.
- [5] CHASE H. LangChain: Building applications with LLMs through composability[EB/OL]. 2023. <https://github.com/langchain-ai/langchain>.
- [6] LIU J. LlamaIndex: A Data Framework for LLM Applications[EB/OL]. 2023. https://github.com/run-llama/llama_index.
- [7] JUMPER J, EVANS R, PRITZEL A, et al. Highly accurate protein structure prediction with AlphaFold[J]. Nature, 2021, 596: 583-589.
- [8] BUTLER K T, DAVIES D W, CARTWRIGHT H, et al. Machine learning for molecular and materials science[J]. Nature, 2018, 559: 547-555.
- [9] HARRIS C R, MILLMAN K J, van der WALT S J, et al. Array programming with NumPy [J]. Nature, 2020, 585: 357-362.
- [10] MCKINNEY W. Data Structures for Statistical Computing in Python[C]//Proceedings of the 9th Python in Science Conference (SciPy). 2010: 56-61.
- [11] WANG L, XU W, LAN Y, et al. Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models[J]. arXiv preprint arXiv:2305.04091, 2023.
- [12] OpenAI. Function Calling and Other API Updates[EB/OL]. 2023. <https://openai.com/blog/function-calling-and-other-api-updates>.
- [13] SCHICK T, DWIVEDI-YU J, DESSI R, et al. Toolformer: Language Models Can Teach Themselves to Use Tools[C]//Advances in Neural Information Processing Systems (NeurIPS). 2023.
- [14] SHEN Y, SONG K, TAN X, et al. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face[C]//Advances in Neural Information Processing Systems (NeurIPS). 2023.
- [15] WEI J, WANG X, SCHUURMANS D, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models[C]//Advances in Neural Information Processing Systems (NeurIPS). 2022.
- [16] WANG X, WEI J, SCHUURMANS D, et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models[C]//International Conference on Learning Representations (ICLR). 2023.
- [17] YAO S, YU D, ZHAO J, et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models[C]//Advances in Neural Information Processing Systems (NeurIPS). 2023.
- [18] SHINN N, CASSANO F, BERMAN E, et al. Reflexion: Language Agents with Verbal Reinforcement Learning[C]//Advances in Neural Information Processing Systems (NeurIPS). 2023.
- [19] PATIL S G, ZHANG T, WANG X, et al. Gorilla: Large Language Model Connected with Massive APIs[J]. arXiv preprint arXiv:2305.15334, 2023.
- [20] QIN Y, LIANG S, YE Y, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs[C]//International Conference on Learning Representations (ICLR). 2024.
- [21] LIU Y, YU L, WANG Y, et al. ReTool: Reinforcement Learning for Strategic Tool Use in LLMs[J]. arXiv preprint, 2024.
- [22] YAN F, MAO H, JI C C J, et al. Berkeley Function Calling Leaderboard[EB/OL]. 2024. <https://gorilla.cs.berkeley.edu/leaderboard.html>.

- [23] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]//Advances in Neural Information Processing Systems (NeurIPS). 2020.
- [24] KARPUKHIN V, OĞUZ B, MIN S, et al. Dense Passage Retrieval for Open-Domain Question Answering[J]. arXiv preprint arXiv:2004.04906, 2020.
- [25] VIRTANEN P, GOMMERS R, OLIPHANT T E, et al. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python[J]. Nature Methods, 2020, 17: 261-272.
- [26] HUNTER J D. Matplotlib: A 2D Graphics Environment[J]. Computing in Science & Engineering, 2007, 9(3): 90-95.
- [27] CHEN Z, CHEN S, NING M, et al. ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery[J]. arXiv preprint arXiv:2410.05080, 2024.
- [28] MIALON G, FOURRIER C, SWIFT C, et al. GAIA: A Benchmark for General AI Assistants [J]. arXiv preprint arXiv:2311.12983, 2023.

附录

A 12 工具的完整接口规范

表 A.1 对第四章表 4.1 的 12 个工具补充给出完整的输入参数与关键输出字段说明，便于读者按字段名复现 DAGPlanner 中的 $\${sX.field}$ 占位符。

表 A.1 12 工具的完整接口规范（输入参数 + 输出字段）

工具名	关键输入参数	关键输出字段	用途
matrix_operation	operation (add/sub/mul/inverse/det), matrix_a, matrix_b	result, shape, determinant, real, imag	矩阵基本运算与行列式
numerical	expression, lower, upper, variable	integral, absolute_error, expression, interval	一元函数自适应数值积分
curve_fitting	model (linear/poly2/exponential/power), x, y	model, params, r_squared, formula	多模型曲线拟合
equation_solver	expression, lower, upper, variable	root, residual, expression, interval	区间内方程求根
statistics	values (数组)	count, mean, median, std, variance, min, max, range, q1, q3, iqr	描述性统计量
hypothesis_test	test_type (one_sample_t / two_sample_t), sample_a, sample_b, alpha	test_type, t_statistic, p_value, alpha, reject_null	t 检验等假设检验
linear_regression	x, y	slope, intercept, r_squared, p_value, std_err, formula	一元线性回归

续下页

工具名	关键输入参数	关键输出字段	用途
correlation	x, y, method (pearson/spearman)	method, correlation, p_value, strength, direction	相关性分析
line_chart	x, y, title, xlabel, ylabel, output_path	file_path, width_pixels, height_pixels	折线图
scatter_chart	x, y, title, xlabel, ylabel, output_path	file_path, width_pixels, height_pixels	散点图
bar_chart	labels, values, title, xlabel, ylabel, output_path	file_path, width_pixels, height_pixels	条形图
heatmap	matrix, row_labels, col_labels, title, output_path	file_path, width_pixels, height_pixels	热力图

““

B O1–O4 提示工程规则全文

第四章 表 4.3 给出了 4 条规则的概要，本节给出其在 SYSTEM_PROMPT 中的实际中文措辞，便于读者复用或修订。

B.1 O1: 结果合理性检查

（从 observation 取值前必须执行。）拟合类工具返回的 R^2 ：若 < 0 或 > 1 ，视为工具输出异常。拟合参数的量级：若拟合参数与输入数据主导量级相差超过 2 个数量级（例如数据在 10^3 量级却拟合出 10^0 量级的常数），视为异常。返回值中出现 NaN / Inf / 空值，视为异常。一旦检测到上述异常，下一轮 必须 尝试不同的解决路径；可选方案包括：换一个拟合模型（例如从 exponential 改为 linear / polynomial）；对数据做变换再拟合；换一个工具解决同一目标。在异常被纠正之前，不得将异常工具返回的数值写入最终答复。

B.2 O2: 数值计算规范

涉及 3 个或以上样本点的统计量（求和、均值、中位数、方差、标准差、四分位数、分位数）必须通过 `descriptive_statistics` 工具计算。严禁在 `thought / content` 里以任何形式写出 “ $(a+b+c+...)/n$ ”、“ $\text{mean} = ...$ ”、“ $\text{sum} = ...$ ” 等手算表达式。线性代数运算（矩阵求逆、乘积、行列式、特征值、转置后再乘等）必须通过 `matrix_operation`。积分、方程求根、曲线拟合 必须通过对应专业工具。最终答复报告数字时，至少保留工具返回原值的 4–6 位有效数字。

B.3 O3: 开场规划模板

对于需要 3 步及以上的任务，在第一轮回复的 `content` 中 必须先以下面的格式列出计划：

Plan:

1. 用 < 工具名 > 做 < 子目标 >, 得到 < 中间变量 >
2. 用 < 工具名 > 做 < 子目标 >, 用到 < 中间变量 >
- ...

写完 Plan 后，同一轮直接发起第一个 `tool_call`。后续每一轮继续推进 Plan：在思考：前缀里简要引用当前走到 Plan 的第几步。若在执行中发现 Plan 有偏差，明确修正 Plan 后再执行。

B.4 O4: Few-shot 示例注入

O4 在 `SYSTEM_PROMPT` 末尾追加一条与本题集无关、用于演示正确工作风格的完整范例（异常切换 + 最终答复格式）。范例本身较长，完整内容见 `src/prompts/system_prompts.py` 中的 `FEWSHOT_EXAMPLE` 常量。

C ext 压力测试用例清单

第五章测试集中的 8 道 `ext_*` 压力题专门用于检验 RATS 与 DAGPlanner 的算法贡献，其设计目标见表 C.1。

表 C.1 8 道 ext_* 压力测试用例的设计目标

编号	用例 ID	设计目标	主要类别
1	ext_rs_01	在 descriptive_statistics 与 hypothesis_test 之间区分（统计描述 vs 假设检验）	statistics
2	ext_rs_02	在 linear_regression 与 correlation_analysis 之间区分（拟合 vs 相关）	statistics
3	ext_rs_03	在 equation_solver 与 numerical_integration 之间区分	numerical
4	ext_rs_04	在 heatmap 与 bar_chart 之间区分（多维 vs 一维可视化）	visualization
5	ext_dag_01	5 步链式：行列式 → 求根 → 积分 → 统计 → 折线图	composite
6	ext_dag_02	4 步链式：统计 → 相关 → 回归 → 散点图	composite
7	ext_dag_03	3 步链式 + 中间结果分支：拟合 + 残差统计 + 残差直方图	composite
8	ext_dag_04	4 步链式：矩阵特征值 → 排序 → 选取 → 条形图	composite

作者简历

基本信息

- 姓 名：彭超琦
- 学 号：3220105744
- 学 院：浙江大学计算机科学与技术学院
- 专 业：计算机科学与技术
- 入学时间：2022 年 9 月

教育经历

- 2022.09 – 2026.06，浙江大学，计算机科学与技术学院，计算机科学与技术专业，本科

兴趣方向

大语言模型与智能体（Agent）；面向科学场景的 AI 工程实践；系统软件与工具链；C/C++ 与 Python 工程开发。

毕业设计相关产出

- 提出并实现了检索增强工具选择算法 RATS 与结构化 DAG 任务拆解算法 DAGPlanner。
- 完成 12 工具的标准化科学计算工具库与覆盖三维判分（数值/文件/工具覆盖）的端到端评测框架。
- 全部源码与评测产物已纳入 Git 版本管理，关键算法模块由独立单元测试覆盖。

本科生毕业论文（设计）任务书

一、题目：面向科学场景的智能体工具调用优化研究

二、指导教师对毕业论文（设计）的进度安排及任务要求：

进度安排：

时间	工作任务
第 1-2 周	文献调研与技术选型，明确研究边界，完成开题报告
第 3-7 周	设计并实现标准化科学工具库，封装核心计算工具，设计任务拆解 Prompt 模板
第 8-11 周	实现检索增强工具选择算法（RATS）与结构化 DAG 任务规划器，完成系统集成联调与对比实验
第 12-14 周	构建测试数据集，开展真实场景验证，整理实验数据与分析结果
第 15-16 周	撰写并修改毕业论文，整理毕设材料，准备答辩

任务要求：学生需要具备扎实的编程能力，熟练掌握 Python 语言，了解科学计算库如 NumPy、SciPy、Pandas 等的使用，有调用外部 API 和工具集成的经验。需要了解大语言模型的基本原理和应用，熟悉 LangChain、AutoGPT 等智能体框架，能够设计和实现工具调用的 Prompt 工程。具备跨学科的知识背景，对至少一个科学领域（如生物、化学、物理等）有基础了解，能够理解科学问题的特点和需求。

起讫日期 2025 年 6 月 1 日至 2026 年 6 月 3 日

指导教师（签名）_____ 职称 _____

三、系或研究所审核意见：

负责人（签名）_____

年 月 日

本科生毕业论文（设计）考核表

一、指导教师对毕业论文（设计）的评语：

论文面向科学场景下智能体的工具调用问题，提出检索增强的工具选择（RATS）与结构化 DAG 任务规划（DAGPlanner）方法，并辅以提示工程规则。建议进一步扩大工具库与任务规模以增强结论普适性。

指导教师（签名）_____

年 月 日

二、答辩小组对毕业论文（设计）的答辩评语及总评成绩：

成绩比例	中期报告 (10%)	开题报告 (15%)	外文翻译 (5%)	毕业设计质量 及答辩 (70%)	总评成绩
分值	8	13	5		

负责人（签名）_____

年 月 日

学生姓名	彭超琦/3220105744
论文题目	面向科学场景的智能体工具调用优化研究
英文题目	Research on Agent Tool Calling Optimization for Scientific Scenarios
起始时间	2025-06-01 至 2026-06-30
学生毕业论文	
毕业论文	3220105744_彭超琦_毕业设计_面向科学场景的智能体工具调用优化研究.pdf (点击下载)
毕业论文上传时间	2026-05-18 19:53:08
导师一审	
导师审核状态	导师审核通过
导师审核时间	2026-05-21 23:34:56
导师审核意见	论文面向科学场景下智能体的工具调用问题，提出检索增强的工具选择（RATS）与结构化 DAG 任务规划（DAG Planner）方法，并辅以提示工程规则。建议进一步扩大工具库与任务规模以增强结论普适性。
导师评分	66
专家一审	
专家审核状态	专家审核通过
专家审核时间	2026-05-25 14:53:13
专家审核意见	该论文围绕科学场景下智能体工具调用优化展开研究，选题具有较强的理论意义和应用价值。论文针对工具选择困难、长依赖链任务易出错、执行轨迹可解释性不足等问题，提出RATS工具选择算法和 DAG Planner任务拆解算法，技术路线较完整，具有一定创新性。实验结果表明所提方法能够提升工具调用成功率和复杂任务处理能力。论文结构完整，实验分析较充分，达到了本科毕业设计要求。